Numerical Methods

Toni Karvonen
toni.karvonen@lut.fi

Lappeenranta–Lahti University of Technology LUT

March 27, 2026

Contents

1 Floating-Point Numbers **1**
1.1 Numeral Systems . . . 1
1.2 Floating-Point Numbers . . . 2
1.3 IEEE 754 Standard . . . 2
1.4 Rounding and Precision . . . 5
1.5 Numerical Stability . . . 6

2 Error **8**
2.1 Rounding and Truncation Error . . . 8
2.2 Precision of Approximation . . . 9
2.3 Absolute and Relative Error . . . 9
2.4 Big-O Notation . . . 10
2.5 Properties of big-O Notation . . . 12

3 Building Blocks of Numerical Methods **15**
3.1 Taylor Polynomials . . . 15
3.2 Linearisation . . . 17
3.3 Array programming . . . 19

4 Numerical Integration **21**
4.1 Midpoint Rule . . . 21
4.2 Trapezoidal Rule . . . 26
4.3 Simpson's Rule . . . 28

5 Solutions to Systems of Equations **32**
5.1 Newton's Method . . . 33
5.2 Newton's Method for Systems of Equations . . . 36
5.3 Fixed Point Method . . . 43
5.4 Jacobi Method for Linear Systems . . . 50

6 Differential Equations **56**
6.1 Euler's Method . . . 56
6.2 Runge–Kutta Methods . . . 56

7 Optimisation **57**
7.1 Basic Concepts in Optimisation . . . 57
7.2 Gradient Descent . . . 57
7.3 Automatic Differentiation . . . 57

1 Floating-Point Numbers

This section discusses how numbers are represented in a computer. Design choices have to be made: there are infinitely many real numbers yet computational resources are finite.

1.1 Numeral Systems

The familiar *decimal system* system represents numbers in powers of 10. For example, the number 1453.529 expands as

$$1453.529 = 1 \cdot 10^3 + 4 \cdot 10^2 + 5 \cdot 10^1 + 3 \cdot 10^0 + 5 \cdot 10^{-1} + 2 \cdot 10^{-2} + 9 \cdot 10^{-3}.$$

In this *positional notation* the contribution of a digit to the value of a number is determined by its value multiplied by a factor that equals a chosen *base* (10 in the decimal system) raised to a power that depends on the position of the digit. The position is counted from the *decimal separator* with non-negative powers on the left and negative on the right. For example, because 5 is the second digit to the left from the separator, it receives power 1 (because counting starts from zero on the left) and thus contributes $5 \cdot 10^1 = 50$. Similarly, because 9 is the third digit to the right, it receives power -3 and contributes $9 \cdot 10^{-3} = 0.009$.

Definition 1.1 (Positional numeral system)**.** Let $b \geq 2$ be an integer called *base* (or *radix*). A number in base b is written as

$$(a_n a_{n-1} \cdots a_0 . c_1 c_2 \cdots)_b = a_n \cdot b^n + a_{n-1} \cdot b^{n-1} + \cdots + a_0 \cdot b^0 + c_1 \cdot b^{-1} + c_2 \cdot b^{-2} + \cdots,$$

where the *digits* $a_0, \ldots, a_n$ and $c_1, c_2, \ldots$ are integers between 0 and $b-1$.

The subscript b is typically not used for decimal numbers (i.e., when $b = 10$) except for emphasis.

Example 1.2. To convert the octal (i.e., base $b = 8$) number $(35.07)_8$ to decimal we expand

$$(35.07)_8 = 3 \cdot 8^1 + 5 \cdot 8^0 + 0 \cdot 8^{-1} + 7 \cdot 8^{-2} = 3 \cdot 8 + 5 + \frac{0}{8} + \frac{7}{8^2} = 29.109375.$$

Example 1.3. In the hexadecimal (i.e., base $b = 16$) the letters from A to F are used to represent the digits from 10 to 15. To convert the hexadecimal number $(\mathrm{AE.1B})_{16}$ to decimal we expand

$$\begin{aligned}(\mathrm{AE.1B})_{16} &= 10 \cdot 16^1 + 14 \cdot 16^0 + 1 \cdot 16^{-1} + 11 \cdot 16^{-2} \\ &= 10 \cdot 16 + 14 + \frac{1}{16} + \frac{11}{256} \\ &= 174.10546875.\end{aligned}$$

Example 1.4. To convert $(220.1)_3$ to decimal we expand

$$(220.1)_3 = 2 \cdot 3^2 + 2 \cdot 3^1 + 0 \cdot 3^0 + 1 \cdot 3^{-1} = 2 \cdot 9 + 2 \cdot 3 + \frac{1}{3} = 24.333\ldots$$

Observe that a finite representation in one base, such as $(220.1)_3$, need not to finite in a different base.

Example 1.5. To convert the decimal number 33.25 to binary (i.e., base $b = 2$) we expand

$$\begin{aligned}(33.25)_{10} = 32 + 1 + \frac{1}{4} &= 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} \\ &= (100001.01)_2.\end{aligned}$$

Binary system. In the binary system the digits (i.e., 0 and 1) are called binary digits, more commonly referred to as *bits*. For example, the binary number $(110.1)_2 = 6.5$ has four bits. The largest number that can be represented with n bits is

$$(1\underbrace{0\cdots00}_{n \text{ times}})_2 - (1)_2 = (\underbrace{11\cdots11}_{n \text{ times}})_2 = 1 \cdot 2^{n-1} + 1 \cdot 2^{n-2} + \cdots + 1 \cdot 2^1 + 1 \cdot 2^0 = 2^n - 1. \tag{1.1}$$

1.2 Floating-Point Numbers

Although any base can be used to represent any number (possibly in a non-terminating manner), this is not possible in practice due to limited memory. When implementing numbers on a computer one has to choose which numbers to represent. This is done via floating-point numbers.

Definition 1.6 (Floating-point number)**.** A *floating-point number* $a \in \mathbb{R}$ in base b has the form

$$a = s \cdot c \cdot b^e,$$

where $s \in \{-1, 1\}$ is the *sign*, $c \in \mathbb{R}_{\geq 0}$ the *mantissa* (or *significand*), and $e \in \mathbb{Z}$ the *exponent*.

The floating-point representation is not unique. For example, 123.45 can be written as

$$12345 \cdot 10^{-2} \quad \text{or} \quad 1234.5 \cdot 10^{-1},$$

or in infinitely many other ways as a decimal (i.e., base $b = 10$) floating-point number. In order to represent floating-point numbers on a computer one must select ranges for the mantissa and exponent. These choices determine the precision and memory cost of the resulting floating-point format.

1.3 IEEE 754 Standard

The *IEEE Standard for Floating-Point Arithmetic* (IEEE 754) defines a collection of floating-point formats used in virtually all computing. The binary IEEE 754 formats represent floating-point numbers in the form

$$a = s \cdot (1 + f) \cdot 2^{e - e_{\text{bias}}}, \tag{1.2}$$

where e_{bias}, the *exponent bias*, is a positive integer and $f \in (0, 1)$ the *fraction*. Both $f \in (0, 1)$ and $e \in \mathbb{Z}_{\geq 0}$ are expressed in binary with chosen numbers of bits. That is,

$$f = (0.\bar{b}_1 \bar{b}_2 \cdots \bar{b}_p)_2 \quad \text{and} \quad e = (\hat{b}_1 \hat{b}_2 \cdots \hat{b}_q)_2$$

for some positive integeres p and q and bits $\bar{b}_i, \hat{b}_j, \in \{0, 1\}$. The sign is represented by a sign b_s in the form $s = (-1)^{b_s}$, so that

$$b_s = 0 \implies s = 1 \qquad \text{and} \qquad b_s = 1 \implies s = -1.$$

To represent a floating-point number in this format the $n = p + q + 1$ bits (p bits for the fraction, q for the exponent and one for the sign) are ordered to give an IEEE 754 binary representation of a floating-point number:

$$(b_s \hat{b}_q \hat{b}_{q-1} \cdots \hat{b}_1 \bar{b}_p \bar{b}_{p-1} \cdots \bar{b}_1)_2 = (b_{p+q} b_{p+q-1} \cdots b_0)_2. \tag{1.3}$$

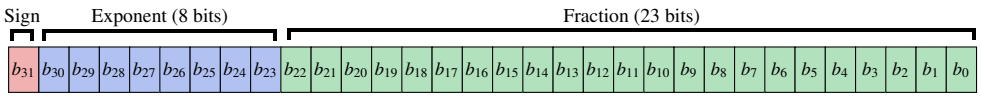


Figure 1.1: Bit layout for 32-bit IEEE 754 floats. See Table 1.1.

Figure 1.1 illustrates the bit scheme. In IEEE 754 the integer e_{bias} in (1.2) that determines the *actual exponent* of two is set as $e_{\text{bias}} = 2^{q-1} - 1$. This means that the actual exponent ranges [recall (1.1)]

$$\text{from} \quad e_{\min} = 0 - e_{\text{bias}} = -2^{q-1} + 1 \quad \text{to} \quad e_{\max} = 2^q - 1 - e_{\text{bias}} = 2^{q-1}(2-1) = 2^{q-1}. \tag{1.4}$$

Example 1.7. Consider IEEE 754 format floating-point numbers with $p = 3$ bits for the fraction and $q = 2$ bits for the exponent. The total number of bits is $n = p + q + 1 = 6$. The binary number $(101110)_2$ represents a floating-point number of the form (1.2) with $s = (-1)^1 = -1$, $e = (01)_2 = 1$, and $f = (0.110)_2 = 0.75$. Because $e_{\text{bias}} = 2^{q-1} - 1 = 1$, $(101110)_2$ corresponds to the decimal number

$$s \cdot (1+f) \cdot 2^{e - e_{\text{bias}}} = (-1) \cdot (1 + 0.75) \cdot 2^{1-1} = -1.75.$$

by (1.2).

Special quantities in IEEE 754. There are exceptions related to the exponents $e = (0 \cdots 0)_2$ and $e = (1 \cdots 1)_2$:

- If the exponent is zero, $e = (0 \cdots 0)_2$:
  - If the *fraction is zero*, $f = (0.0 \cdots 0)_2$, the number is interpreted as zero. The sign bit determines the whether the zero is "positive" or "negative", which has no effect on most computations. That is,

    $$\text{“positive” zero} = (10 \cdots 0)_2 \quad \text{and} \quad \text{“negative” zero} = (00 \cdots 0)_2.$$

  - If the *fraction is not zero*, the floating-point number is called *subnormal* and represents a decimal number of the form

    $$a_{\text{subnormal}} = s \cdot f \cdot 2^{-e_{\text{bias}}+1}, \tag{1.5}$$

    which differs from a *normal* number of the form (1.2) by $1 + f \in (1,2)$ being replaced by $f \in (0,1)$. Smaller numbers than what would be possible with normal numbers alone can be represented with subnormal numbers.
- If the exponent is maximal, $e = (1 \cdots 1)_2$:
  - If the *fraction is zero*, $f = (0.0 \cdots 0)_2$, the number is interpreted as positive (∞) or negative infinite ($-\infty$) depending on the sign bit.
  - If the *fraction is not zero*, the number is interpreted as *not a number* (NaN), meaning that the number is undefined. This can occur when trying to compute $0/0$.

IEEE 754 binary formats. Table 1.1 contains the specifications and key properties of the three most common IEEE 754 binary floating-point formats. These are typically referred to as *half-precision* numbers (16 bit), *floats* or *single-precision numbers* (32 bits), and *doubles* or *double-precision numbers* (64 bits).

Table 1.1: Key properties of IEEE 754 half, single-precision float, and double-precision formats. Note that each format includes one bit for the sign in addition to the exponent and fraction bits. *Machine epsilon* quantifies the precision of a floating-point system. It refers to the difference between 1 and the next larger floating-point number. One can work out that $\varepsilon = 2^{-p}$, where p is the number of bits in the fraction. *The number bits in the fraction determines precision and the number of bits in the exponent determines how small and large numbers can be represented.*

Property	**Half (16-bit)**	**Float (32-bit)**	**Double (64-bit)**
Total bits	16	32	64
Exponent bits (q)	5	8	11
Fraction bits (p)	10	23	52
Exponent bias (e_{bias})	15	127	1023
Smallest positive normal	2^{-14}	2^{-126}	2^{-1022}
Smallest positive subnormal	2^{-24}	2^{-149}	2^{-1074}
Largest finite value ($\approx$)	6.55×10^4	3.40×10^{38}	1.80×10^{308}
Machine epsilon (ε)	2^{-10}	2^{-23}	2^{-52}

Example 1.8. Let us find the IEEE 754 float (32 bits) representation of the decimal number $a = -3.75$. Since the sign is $s = (-1)^{b_s}$, the sign bit is one, $b_s = 1$. By (1.4), the exponent bias is $e_{\text{bias}} = 2^{q-1} - 1 = 2^7 - 1 = 127$. Because $1 + f \in (1,2)$ and $(1+f) \cdot 2^1 \in (2,4)$, the actual exponent must be 1, meaning that $1 = e - e_{\text{bias}} = e - 127$. Therefore $e = 128 = 2^7 = (10\cdots0)_2$, where there are seven zeros. From $3.75 = (1+f) \cdot 2$ we solve $f = \frac{1}{2} \cdot 3.75 - 1 = 0.875 = (0.111)_2$. Therefore the float representation of -3.75 is

$$11000000011100000000000000000000,$$

where we use the same colours as in Figure 1.1 to represent the sign, exponent, and fraction, and have appended zeros to the fraction to ensure that the representation has 32 bits.

Example 1.9. Let us proceed to the opposite direction and find out which decimal number a the float represention

$$00001010001100000000000000000000$$

corresponds to. The sign bit is zero ($b_s = 0$), so that the sign is positive because $s = (-1)^{b_s} = (-1)^0 = 1$. The exponent is $e = (00010100)_2 = 2^4 + 2^2 = 20$. The fraction is $f = (0.01100000000000000000000)_2 = 2^{-2} + 2^{-3} = 0.375$. Therefore

$$a = s \cdot (1+f) \cdot 2^{e - e_{\text{bias}}} = 1 \cdot (1 + 0.375) \cdot 2^{20-127} = 1.375 \cdot 2^{-107} \approx 8.47 \times 10^{-33}.$$

Example 1.10. Let us verify some properties of floats in Table 1.1. By (1.4), the exponent bias is $e_{\text{bias}} = 2^{q-1} - 1 = 2^7 - 1 = 127$. The smallest possible positive normal number is the smallest positive floating-point number of the form (1.2). This means that we want (a) a fraction that is as small as possible and (b) a *positive* exponent (if the exponent were zero

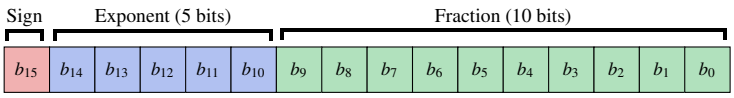


Figure 1.2: Bit layout for 16-bit IEEE 754 half-precision floating point numbers. See Table 1.1.

the number would be subnormal) that is as small as possible. This means that

$$f = (0.\underbrace{0\cdots0}_{23\text{ times}})_2 = 0 \quad \text{and} \quad e = (\underbrace{0\cdots0}_{7\text{ times}}1)_2 = 1.$$

Therefore the smallest positive *normal* float is

$$a = 1\cdot(1+f)\cdot 2^{e-e_{\text{bias}}} = 1\cdot(1+0)\cdot 2^{1-127} = 2^{-126} \approx 1.18\times 10^{-38}.$$

To get the smallest positive *subnormal* float we set the exponent to zero, select the smallest positive fraction, $f = (0.0\cdots01)_2 = 2^{-23}$ and use the subnormal form (1.5), which gives

$$a = 1\cdot f\cdot 2^{-127+1} = 2^{-23}\cdot 2^{-126} = 2^{-149} \approx 1.40\times 10^{-45}.$$

The largest finite float has largest possible fraction and an exponent that is one minus the largest possible exponent (because the largest exponent is interpreted as infinity or NaN):

$$f = (0.\underbrace{1\cdots1}_{23\text{ times}})_2 = \sum_{k=1}^{23} 2^{-k} = 1-2^{-23} \quad \text{and} \quad e = (\underbrace{1\cdots1}_{7\text{ times}}0)_2 = \sum_{k=1}^{7} 2^{k} = 254.$$

These give the largest finite float

$$a = 1\cdot(1+f)\cdot 2^{e-e_{\text{bias}}} = (2-2^{-23})\cdot 2^{127} \approx 3.40\times 10^{38}.$$

Floating-point numbers in Python. In Python, one way to recover the 32-bit binary representation of a floating-point number is as follows:

```
import numpy as np
a = -3.75
res = np.binary_repr(np.float32(a).view(np.int32), width=32)
print(res)
```

Running these lines of code should print `11000000011100000000000000000000`, which matches with the representation we obtained in Example 1.8. However, note that in Python the term "float" refers to double-precision (64-bit) floating point numbers.

1.4 Rounding and Precision

Floating-point numbers provide a *finite-precision arithmetic*. That is, not all real numbers can be represented exactly and the results of arithmetic operations can differ from *exact arithmetic*. For example, the familiar decimal number 0.1 cannot be represented exactly as a non-terminating binary number:

$$(0.1)_{10} = (0.00011001100110011\ldots)_2.$$

Consequently, such numbers cannot be represented exactly as binary floating-point numbers and are rounded to a representable floating-point number according the rounding rules specified in the IEEE 754 standard. This can be readily observed in computations. For example, in Python

a simple multiplication whose value in exact arithmetic is 653082380.34 becomes something different:

```
>>> print(27869.01 * 23434)
653082380.3399999
```

We can observe the same phenomenon when printing, for example, 50 decimal digits of $a = 0.1$:

```
>>> a = 0.1
>>> print(f"{a:.{50}f}")
0.10000000000000000555111512312578270211815834045410
```

Precision. Floating-point numbers are *not uniform in precision*. This is both an advantage (a huge range of numbers can be represented) and a disadvantage (rounding errors are inevitable when operating with large numbers). For example, with doubles (64 bits) one can represent numbers between 1 and 2 with precision 2^{-52}, meaning that all numbers on the interval $[1,2]$ round to a multiple of 2^{-52}. Precision decreases as numbers become larger, being 2^{-51} for numbers between 2 and 4 and eventually 2^{971} for numbers between 2^{1023} and 2^{1024}. We can easily see this in action in Python (which uses doubles by default). The following computations return positive values as they should:

```
>>> b = 1.0
>>> a = 1.0 + 2.0**(-52)
>>> print(a - b)
2.220446049250313e-16
>>> b = 2.0**1023
>>> a = 2.0**1023 + 2.0**971
>>> print(a - b)
1.99584030953472e+292
```

Note that in the second computation it is essential to write "`2.0`" rather than "2" to ensure that Python interprets the numbers as floating-point numbers rather than integers, which are handled differently. However, if we make a slightly smaller it is rounded to b and the difference is incorrectly (in comparison to exact arithmetic) evaluated as zero:

```
>>> b = 1.0
>>> a = 1.0 + 0.5*2.0**(-52)
>>> print(a - b)
0.0
>>> b = 2.0**1023
>>> a = 2.0**1023 + 0.5*2.0**971
>>> print(a - b)
0.0
```

Underflow and overflow. Similar phenomena occur when one tries to do arithmetic with numbers smaller than the *underflow level* (the row with smallest positive subnormals in Table 1.1) rounding to zero and numbers larger than the *overflow level* (the row with largest finite values in Table 1.1) rounding to infinity or throwing an error. For 64-bit doubles these levels are 2^{-1074} and approximately 1.80×10^{308}.

1.5 Numerical Stability

The inherent inadequacies of floating-point numbers need to be taken into account when designing numerical algorithms. *Numerical stability* of an algorithm refers to its ability to tolerate numerical

errors. One should be particularly careful when subtracting nearby floating-point numbers and avoid doing this if possible.

Example 1.11. Computing $\sqrt{1+x}-\sqrt{x}>0$ is prone to numerical cancellations when x is large because in that case both $\sqrt{1+x}$ and $\sqrt{x}$ are large and nearby but far away from nearest representable floating-point numbers due to their sparsity (recall Section 1.4). For example, for $x=2^{60}$ this expression evaluates to zero in the 64-bit arithmetic used in Python. In contrast, the equivalent form

$$\sqrt{1+x}-\sqrt{x}=\frac{(\sqrt{1+x}-\sqrt{x})(\sqrt{1+x}+\sqrt{x})}{\sqrt{1+x}+\sqrt{x}}=\frac{1}{\sqrt{1+x}+\sqrt{x}},$$

which used the identity $(a-b)(a+b)=a^2-b^2$, evaluates to a positive value even for $x=2^{1023}$.

Example 1.12. Similar phenomenon can occur when computing a^2-b^2 for large a and b. This computation can be made more stable by using the identity $(a-b)(a+b)=a^2-b^2$. While this identity contains a subtraction $a-b$, the numbers being subtracted are smaller than in a^2-b^2 whenever $a,b>1$. Therefore floating-point rounding is less extreme.

The following lines of Python illustrate Example 1.12:

```
>>> a = 2.0**50
>>> b = a - 1.0
>>> print(a**2 - b**2)
2251799813685248.0
>>> print((a - b)*(a + b))
2251799813685247.0
```

The correct value of a^2-b^2 for $a=2^{50}$ and $b=2^{50}-1$ is 2251799813685247, which the computation based on the more numerically stable form $(a-b)(a+b)$ returned.

2 Error

This section discusses the nature of numerical errors in computation and how errors are measured.

2.1 Rounding and Truncation Error

Numerical errors arise from both the inherent inaccuracy of finite-precision arithmetic and numerical approximations of mathematical processes.

Definition 2.1 (Rounding error)**.** *Rounding error* is the error caused by the use of finite-precision arithmetic (e.g., floating-point numbers) as opposed to exact arithmetic.

Example 2.2. The rounding and precision problems that we observed in Sections 1.4 and 1.5 are examples of rounding errors.

Definition 2.3 (Truncation error)**.** *Truncation error* is the error caused by a numerical approximation of a mathematical process.

Example 2.4. From basic analysis we recall that the exponential constant $e \approx 2.718$ can be expressed as the infinite series

$$e = \sum_{k=0}^{\infty} \frac{1}{k!},$$

where $n! = 1 \cdot 2 \cdots (n-1) \cdot n$ is the factorial and $0! = 1$. To approximate e one can truncate this series after n terms (here $n \in \{0,1,2,3\}$):

$$\begin{aligned}
e \approx e_0 &= \sum_{k=0}^{0} \frac{1}{k!} = \frac{1}{0!} = \frac{1}{1} = 1, \\
e \approx e_1 &= \sum_{k=0}^{1} \frac{1}{k!} = \frac{1}{0!} + \frac{1}{1!} = \frac{1}{1} + \frac{1}{1} = 2, \\
e \approx e_2 &= \sum_{k=0}^{2} \frac{1}{k!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} = \frac{1}{1} + \frac{1}{1} + \frac{1}{2} = 2.5, \\
e \approx e_3 &= \sum_{k=0}^{3} \frac{1}{k!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} = \frac{1}{1} + \frac{1}{1} + \frac{1}{2} + \frac{1}{2 \cdot 3} = 2.666\ldots
\end{aligned}$$

Any of the differences $e - e_n$ for $n \in \{0,1,2,3\}$ is a truncation error.

However, truncation error does not refer only to explicit truncation as in Example 2.4 but to any type of numerical approximation.

Example 2.5. Recall again from basic analysis that the derivative of a function $f \colon \mathbb{R} \to \mathbb{R}$ at a point $x_0 \in \mathbb{R}$ is defined as the limit

$$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h}$$

of the difference quotient $[f(x_0 + h) - f(x_0)]/h$. One can therefore approximate the derivative by computing the difference quotient for a small h. For example, the derivative of

$f(x) = x^2$ at $x_0 = 1$ can be approximated as (with $h = 1/10$)

$$f'(x_0) \approx \frac{f(x_0+h) - f(x_0)}{h} = \frac{(1+1/10)^2 - 1^2}{1/10} = \frac{21}{10}.$$

Since $f'(x) = 2x$, the true derivative is $f'(x_0) = f'(1) = 2$. The error

$$f'(x_0) - \frac{21}{10} = 2 - \frac{21}{10} = -\frac{1}{10}$$

is a truncation error even though no summation is being truncated. However, even here one can think in terms of a truncation: one is "truncating the limiting process $h \to 0$" by picking a small h and using that rather than going all the way to zero.

2.2 Precision of Approximation

Definition 2.6 (Precision of approximation)**.** An approximation $\tilde{x}$ to x is said to *approximate x to d decimal places* if

$$|x - \tilde{x}| < \frac{1}{2}10^{-d}.$$

For example, if we want $\tilde{x}$ to approximate x to the nearest integer (i.e., to zero decimal places), then it must hold that $|x - \tilde{x}| < 0.5$. This means that the distance between x and $\tilde{x}$ is always rounded to zero when approximated to the nearest integer. Note that this does not mean that both numbers are rounded to the same value when approximated to the nearest integer. For example, if $x = 0.51$ and $\tilde{x} = 0.49$, then $|0.51 - 0.49| = 0.02 < 0.5$ but $x \approx 1$ and $\tilde{x} \approx 0$. Note also that if $\tilde{x}$ approximates x to d decimal places, it also approximates it to $d-1$ decimal places, and so on. What matters is, of course, the highest decimal precision.

Example 2.7. The number 3.14 approximates π to two decimal places since

$$|\pi - 3.14| = 0.00159\ldots = 0.159\ldots \cdot 10^{-2} < 0.5 \cdot 10^{-2}.$$

The number 3.14 approximates π also to one decimal place since

$$|\pi - 3.14| = 0.00159\ldots = 0.0159\ldots \cdot 10^{-1} < 0.5 \cdot 10^{-1}.$$

The numbe 3.14 does *not* approximate π to three decimal places since

$$|\pi - 3.14| = 0.00159\ldots = 1.59\ldots \cdot 10^{-3} > 0.5 \cdot 10^{-3}.$$

2.3 Absolute and Relative Error

Error can be measured in either absolute or relative terms.

Definition 2.8 (Absolute and relative error)**.** Let $\tilde{x}$ be an approximation of x. Then

$$\textit{absolute error} = |x - \tilde{x}| \quad \text{and} \quad \textit{relative error} = \left|\frac{x - \tilde{x}}{x}\right|.$$

The absolute error is simply the magnitude of the difference between the true quantity of interest x and its approximation $\tilde{x}$. The relative error measures the error *in proportion to the magnitude of the true value*. Note that when computing the relative error one divides by the magnitude of the true value, not of the approximation. The relative error is often expressed in percent.

Example 2.9. If the number $x = \pi$ is approximated to two decimal places as in Example 2.7 (i.e., $\tilde{x} = 3.14$), then the absolute error of the approximation is

$$|\pi - 3.14| = 0.00159\ldots$$

and the relative error is

$$\left|\frac{\pi - 3.14}{\pi}\right| = 0.000507\ldots = 0.0507\ldots\,\%.$$

Relative error is often the more useful of the two errors.

Example 2.10. In Section 1.4 we observed that in Python's 64-bit double-precision arithmetic the number

$$x = 2^{1023} + 0.5 \cdot 2^{971} \quad \text{is rounded to} \quad \tilde{x} = 2^{1023}.$$

Therefore the absolute rounding error is

$$|x - \tilde{x}| = |2^{1023} + 0.5 \cdot 2^{971} - 2^{1023}| = 0.5 \cdot 2^{971} \approx 9.9792 \cdot 10^{291}.$$

This somewhat large number is very small in proportion to the size of x as the relative rounding error is minuscule:

$$\left|\frac{x - \tilde{x}}{x}\right| = \left|\frac{2^{1023} + 0.5 \cdot 2^{971} - 2^{1023}}{2^{1023} + 0.5 \cdot 2^{971}}\right| \approx 1.11 \cdot 10^{-16} = 1.11 \cdot 10^{-14}\,\%.$$

2.4 Big-O Notation

Asymptotic big-O notation provides a convenient and concise way to describe the asymptotical, or limiting, behaviour of an approximation.

Definition 2.11 (Big-O notation for functions)**.** Let $x_0 \in (a,b)$ and let $f, g\colon (a,b) \to \mathbb{R}$ be functions. We write

$$f(x) = O(g(x)) \quad \text{as} \quad x \to x_0$$

if there are $C \geq 0$ and $\delta > 0$ such that $|f(x)| \leq Cg(x)$ for all $x \in (x_0 - \delta, x_0 + \delta)$.

Recall that a sequence $(a_n)_{n=1}^{\infty} = (a_1, a_2, \ldots)$ is an ordered collection of real numbers.

Definition 2.12 (Big-O notation for sequences)**.** Let $(a_n)_{n=1}^{\infty}$ and $(b_n)_{n=1}^{\infty}$ be two sequences. We write

$$a_n = O(b_n) \quad \text{as} \quad n \to \infty$$

if there is $C \geq 0$ such that $|a_n| \leq Cb_n$ for all $n \geq 1$.

The notation $O(g(x))$ is read as "of order $g(x)$", "big-oh of $g(x)$", or "oh of $g(x)$". If clear from the context, the limit ($x \to x_0$ or $n \to \infty$) is often omitted. On this course we use big-O notation to describe the behaviour of numerical approximations. However, this notation is very versatile, being also used to describe the time complexity and space complexity of algorithms.[1] If $g(x) \to 0$ as $x \to x_0$ (or $b_n \to 0$ as $n \to \infty$), the statement $f(x) = O(g(x))$ gives information on how fast $f(x)$ tends to zero as $x \to x_0$. Namely, this means that $f(x)$ tends to zero *at least* as fast as $g(x)$. In such a situation one wants to find g that tends to zero as fast as possible such that $f(x) = O(g(x))$ remains true.

[1] *Data Structures and Algorithms* (BM40A1500).

Example 2.13. Suppose that we want to determine the order of $f(x) = x^2 + x^3$ as $x \to 0$ (i.e., $x_0 = 0$). By Definition 2.11 we need to find a function g and constants $C \geq 0$ and $\delta > 0$ such that $|f(x)| \leq Cg(x)$ for all $x \in (-\delta, \delta)$. Select $\delta = 1$ (any other value less than one would do as well). Then, for all $x \in (-1, 1)$,

$$|f(x)| = x^2 + |x|^3 \leq x^2 + |x|^2 = 2x^2,$$

where we used the fact that $|a|^n \leq |a|^m$ if $|a| \leq 1$ and $n \geq m$. Therefore we can select $C = 2$ and $g(x) = x^2$, meaning that $f(x)$ is of order x^2 [$f(x) = O(x^2)$] as $x \to 0$. However, note that we also have

$$|f(x)| = x^2 + |x|^3 \leq x^2 + |x|^2 \leq |x| + |x| = 2|x|,$$

so that we could have chosen $g(x) = |x|$.

In Example 2.13 the order $g(x) = x^2$ is better than $g(x) = |x|$ because it describes the behaviour of f close to $x_0 = 0$ more accurately. Namely, as $x \to 0$,

$$f(x) = O(x^2) \quad \text{and} \quad f(x) = O(|x|)$$

but

$$x^2 = O(|x|) \quad \text{and} \quad |x| \neq O(x^2),$$

where $|x| \neq O(x^2)$ means that $|x|$ is *not* of order x^2 as $x \to 0$. When written down these four big-O statements are

$$f(x) \text{ tends to zero at least as fast as } x^2 \quad \text{and} \quad f(x) \text{ tends to zero at least as fast } |x|$$

but

$$x^2 \text{ tends to zero at least as fast as } |x| \quad \text{and} \quad |x| \text{ tends to zero } \textit{slower} \text{ than } x^2.$$

Figure 2.1 illustrates big-O notation for common polynomials around $x_0 = 0$.

Remark 2.14. To write $f(x) = O(g(x))$ as $x \to x_0$ is an abuse of notation. Because "=" commutes elsewhere in mathematics (i.e., $a = b$ is the same as $b = a$), $f(x) = O(g(x))$ suggests that one could write $O(g(x)) = f(x)$, which is difficult to make sense of. What $f(x) = O(g(x))$ as $x \to x_0$ means is that f *is an element of the collection of functions that*

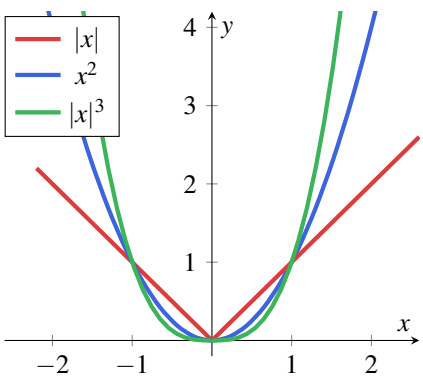


Figure 2.1: The functions $|x|$, x^2 and $|x|^3$ around $x_0 = 0$. Because $|x|$ is above x^2 when $x \in (-1, 1)$ and x^2 is above $|x|^3$, the figure illustrates that $x^2 = O(|x|)$ and $|x|^3 = O(x^2)$ as $x \to 0$. *Observe that the ordering of the graphs changes when* $|x| > 1$. If we considered a point x_0 such that $|x_0| > 1$ (say $x_0 = 2$), the big-O relations would need to be reversed: $|x| = O(x^2)$ and $x^2 = O(|x|^3)$ as $x \to 2$. Note that here we must use $|x|$ and $|x|^3$ rather than x and x^3 because x and x^3 are negative when $x < 0$ but $f(x) = O(g(x))$ enforces an upper bound on $|f(x)| \geq 0$ rather than $f(x)$.

are bounded from above by some constant times $g(x)$ around x_0. The notation must therefore be interpreted as convenient shorthand only.

Sometimes you encounter the notation

$$f(x) = g(x) + O(r(x)) \quad \text{as} \quad x \to x_0.$$

This notation means that f is the sum of g and a term that is of order $r(x)$ as $x \to x_0$. For example, if $f(x) = 3 + x^2 + x^3$, we could write

$$f(x) = 3 + O(x^2) \quad \text{as} \quad x \to 0$$

because in Example 2.13 we saw that $x^2 + x^3 = O(x^2)$ as $x \to 0$.

2.5 Properties of big-O Notation

The following theorem shows that big-O notation is *transitive*. The theorem is nothing but a big-O version of the elementary fact that $a \le b$ and $b \le c$ implies $a \le c$.

Theorem 2.15. *If $f(x) = O(g_1(x))$ and $g_1(x) = O(g_2(x))$ as $x \to x_0$, then*

$$f(x) = O(g_2(x)) \quad \textit{as} \quad x \to x_0.$$

Proof. By Definition 2.11, there are $C_1, C_2 \ge 0$ and $\delta_1, \delta_2 > 0$ such that

$$|f(x)| \le C_1 g_1(x) \quad \text{for all} \quad x \in (x_0 - \delta_1, x_0 + \delta_1)$$

and

$$|g_1(x)| \le C_2 g_2(x) \quad \text{for all} \quad x \in (x_0 - \delta_2, x_0 + \delta_2).$$

These two inequalities are valid simultaneously when $x \in (x_0 - \delta, x_0 + \delta)$ for $\delta = \min\{\delta_1, \delta_2\}$. Therefore, for $x \in (x_0 - \delta, x_0 + \delta)$,

$$|f(x)| \le C_1 g_1(x) \le C_1 |g_1(x)| \le C_1 \cdot C_2 g_2(x),$$

so that we can set $g = g_2$, $C = C_1 \cdot C_2$ and $\delta = \min\{\delta_1, \delta_2\}$ in Definition 2.11. □

The next theorem shows that the constant C in Definition 2.11 does not affect the order.

Theorem 2.16. *If $f(x) = O(g_1(x))$ as $x \to x_0$ and $g_1(x) = C_{12} g_2(x)$ for $C_{12} \ge 0$, then $f(x) = O(g_2(x))$ as $x \to x_0$.*

Proof. By Definition 2.11 there are $C_1 \ge 0$ and $\delta > 0$ such that $|f(x)| \le C_1 g_1(x)$ for all $x \in (x_0 - \delta, x_0 + \delta)$. From the assumption $g_1(x) = C_{12} g_2(x)$ it follows that, for all $x \in (x_0 - \delta, x_0 + \delta)$,

$$|f(x)| \le C_1 g_1(x) = C_1 \cdot C_{12} g_2(x),$$

so that we can select $C = C_1 \cdot C_{12}$ and $g = g_2$ in Definition 2.11. □

Theorems 2.15 and 2.16 have natural variants for sequences (Definition 2.12). The following example shows a rather typical application of big-O notation in numerical approximation where one is usually interested in the behaviour of an error as some *discretisation parameter* h tends to zero.

Example 2.17. In Example 2.5 we approximated the derivative of $f(x) = x^2$ at $x_0 = 1$ with the difference quotient

$$q(h) = \frac{f(x_0 + h) - f(x_0)}{h}.$$

We now compute the order of the absolute error of this approximation as $h \to 0$. Because

$f'(x) = 2x$ and $x_0 = 1$, the absolute error $\varepsilon(h)$ is

$$\begin{aligned}\varepsilon(h) = |f'(x_0) - q(h)| = \left|f'(x_0) - \frac{f(x_0+h) - f(x_0)}{h}\right| &= \left|2 - \frac{(1+h)^2 - 1}{h}\right| \\ &= \left|2 - \frac{2h + h^2}{h}\right| \\ &= |2 - (2+h)| \\ &= |h|,\end{aligned}$$

where we used the identity $(a+b)^2 = a^2 + 2ab + b^2$. This shows that $\varepsilon(h) = O(|h|)$, which means that the absolute error of the derivative approximation by the difference quotient $f'(x_0) \approx q(h)$ tends to zero with *linear order* as $h \to 0$. Note that the relative error $\rho(h)$ is of the same order because

$$\rho(h) = \left|\frac{f'(x_0) - q(h)}{f'(x_0)}\right| = \frac{1}{|f'(x_0)|}\varepsilon(h),$$

where $1/|f'(x_0)| = 1/2 > 0$. This shows that $\rho(h) = O(\varepsilon(h))$. Since $\varepsilon(h) = |h|$, we can now use Theorem 2.16 to deduce that $\rho(h) = O(|h|)$.

We shall mostly work with errors $\varepsilon(h)$ that have *polynomial orders* as $h \to 0$. If a function is a polynomial, the term with the smallest degree determines the order.

Theorem 2.18. *If $\varepsilon(h) = a_n h^n + a_{n+1}h^{n+1} + \cdots + a_{n+m}h^{n+m}$ for $n, m \in \mathbb{N} \cup \{0\}$ and $a_n, \ldots, a_{n+m} \in \mathbb{R}$, then*

$$\varepsilon(h) = O(|h|^n) \quad \text{as} \quad h \to 0.$$

Proof. Select $\delta = 1$ and note that $x_0 = 0$. For $h \in (x_0 - \delta, x_0 + \delta) = (-1, 1)$,

$$|\varepsilon(h)| = |a_n h^n + a_{n+1}h^{n+1} + \cdots + a_{n+m}h^{n+m}| \leq |a_n||h|^n + \cdots + |a_{n+m}||h|^{n+m},$$

where we used the triangle inequality $|a+b| \leq |a| + |b|$. Because $|h|^{n+m} \leq |h|^n$ for $h \in (-1, 1)$, we obtain

$$|\varepsilon(h)| \leq (|a_n| + \cdots + |a_{n+m}|)|h|^n,$$

where the constant $C = |a_n| + \cdots + |a_{n+m}|$ is non-negative (i.e., ≥ 0). It follows from Definition 2.11 that $\varepsilon(h) = O(|h|^n)$. □

We are yet to discuss Definition 2.12, which defines big-O notation for sequences. On this course we use this notation to describe the error of numerical methods as n, the number of *discretisation points* or *time steps*, grows.

Example 2.19. Recall Example 2.4 where we approximated the exponential constant $e \approx 2.718$ by truncating the infinite series expansion $e = \sum_{k=0}^{\infty} 1/k!$ after n terms:

$$e \approx e_n = \sum_{k=0}^{n} \frac{1}{k!}.$$

Let us determine the order of this approximation (i.e., the order of the error $\varepsilon_n = |e - e_n|$) as $n \to \infty$. We have

$$\varepsilon_n = |e - e_n| = \left|\sum_{k=0}^{\infty} \frac{1}{k!} - \sum_{k=0}^{n} \frac{1}{k!}\right| = \left|\sum_{k=n+1}^{\infty} \frac{1}{k!}\right| = \sum_{k=n+1}^{\infty} \frac{1}{k!}. \tag{2.1}$$

For any $m \geq 1$ we can write

$$(n+m)! = 1 \cdot 2 \cdots (n+1) \cdot (n+2) \cdots (n+m) = (n+1)! \cdot (n+2) \cdots (n+m), \tag{2.2}$$

where the product $(n+2) \cdots (n+m)$ is considered equal to one if $m = 1$. By plugging (2.2) in (2.1) we obtain

$$\varepsilon_n = \sum_{k=n+1}^{\infty} \frac{1}{k!} = \sum_{m=1}^{\infty} \frac{1}{(n+m)!} = \frac{1}{(n+1)!} \left(1 + \sum_{m=2}^{\infty} \frac{1}{(n+2) \cdots (n+m)} \right).$$

The final series converges (we skip the proof), which means that we can set

$$a_n = \varepsilon_n, \quad b_n = \frac{1}{(n+1)!} \quad \text{and} \quad C = 1 + \sum_{m=2}^{\infty} \frac{1}{(n+2) \cdots (n+m)}$$

in Definition 2.12. Therefore

$$\varepsilon_n = |e - e_n| = O\left(\frac{1}{(n+1)!} \right).$$

3 Building Blocks of Numerical Methods

This section reviews basic notions and tools that recur when numerical methods are constructed and implemented.

3.1 Taylor Polynomials

The Taylor expansion of a smooth (i.e., sufficiently many times differentiable) function forms the basis of many numerical methods.

Definition 3.1 (Taylor polynomials)**.** Let $f\colon (a,b) \to \mathbb{R}$ be an n times differentiable function at a point $x_0 \in (a,b)$. The nth *Taylor polynomial* of f at the expansion point x_0 is

$$P_n(x) = f(x_0) + f'(x_0)(x-x_0) + \frac{f''(x_0)}{2!}(x-x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x-x_0)^n. \tag{3.1}$$

The nth Taylor polynomial is also called the Taylor polynomial *of degree* n.

Recall that $f^{(n)}(x)$ stands for the nth derivative of f at a point x. That is,

$$f^{(n)}(x) = \frac{\mathrm{d}^n}{\mathrm{d}x^n} f(x).$$

Moreover, the zeroth derivative is simply the function itself: $f^{(0)}(x) = f(x)$. Therefore the summation notation can be used to write the Taylor polynomial as

$$P_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x-x_0)^k.$$

Example 3.2. Let us form the Taylor polynomials of degrees $n \in \{1,3,5\}$ for the function $f(x) = \sin(x)$ at the point $x_0 = 0$. To form these Taylor polynomials we need the values of derivatives of f up to order 5 at $x_0 = 0$. By recalling that the derivative of $\sin(x)$ is $\cos(x)$

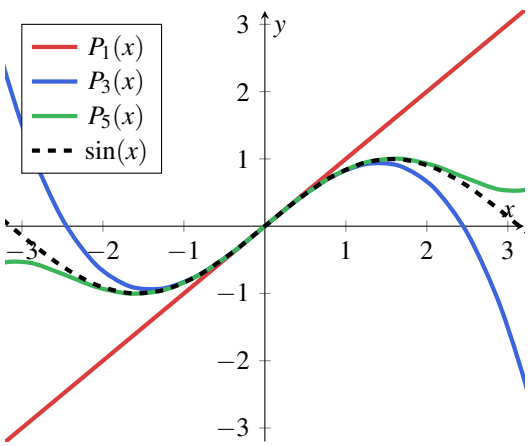


Figure 3.1: The function $f(x) = \sin(x)$ and its Taylor polynomials P_1, P_3 and P_5 of degrees $n \in \{1,3,5\}$. Taylor polynomials are defined in (3.1) and computed for the sinusoid in Example 3.2. Observe that increasing the degree of the Taylor polynomial improves the accuracy of the approximation $f(x) \approx P_n(x)$. The polynomial P_1 is a decent approximation when $|x| < 1$ whereas P_5 works well for $|x| < 2.5$.

and the derivative of $\cos(x)$ is $-\sin(x)$ we get

$$\begin{aligned}
f^{(0)}(x) &= f(x) = \sin(x) &&\implies && f^{(0)}(0) = 0,\\
f'(x) &= \cos(x) &&\implies && f'(0) = 1,\\
f''(x) &= -\sin(x) &&\implies && f''(0) = 0,\\
f^{(3)}(x) &= -\cos(x) &&\implies && f^{(3)}(0) = -1,\\
f^{(4)}(x) &= \sin(x) &&\implies && f^{(4)}(0) = 0,\\
f^{(5)}(x) &= \cos(x) &&\implies && f^{(5)}(0) = 1.
\end{aligned}$$

By plugging these values in (3.1) we obtain the Taylor polynomials

$$\begin{aligned}
P_1(x) &= f(x_0) + f'(x_0)(x - x_0) = f(0) + f'(0)x = x,\\
P_3(x) &= f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f^{(3)}(0)}{3!}x^3 = x - \frac{1}{6}x^3,\\
P_5(x) &= f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f^{(3)}(0)}{3!}x^3 + \frac{f^{(4)}(0)}{4!}x^4 + \frac{f^{(5)}(0)}{5!}x^5\\
&= x - \frac{1}{6}x^3 + \frac{1}{120}x^5.
\end{aligned}$$

These polynomials are depicted in Figure 3.1.

From Example 3.2 and Figure 3.1 we observe that increasing the degree of the Taylor polynomial appears to make it a better approximation to f. Taylor's theorem familiar from basic analysis puts this observation on rigorous footing.

Theorem 3.3 (Taylor's theorem)**.** *Suppose that $f\colon [a,b] \to \mathbb{R}$ is continuous on $[a,b]$ and $n+1$ times differentiable on (a,b). Let $x_0 \in (a,b)$. Then for each $x \in [a,b]$ we have*

$$f(x) - P_n(x) = R_{n+1}(x),$$

where the remainder term R_{n+1} *is given by*

$$R_{n+1}(x) = \frac{f^{(n+1)}(c_x)}{(n+1)!}(x - x_0)^{n+1}$$

for some c_x between x_0 and x.

The number c_x in Taylor's theorem depends on x and n. We can use big-O notation from Section 2.4 to rewrite Taylor's theorem if the $(n+1)$th derivative of f is uniformly bounded.

Corollary 3.4. *Suppose that $f\colon [a,b] \to \mathbb{R}$ is continuous on $[a,b]$ and $n+1$ times differentiable on (a,b). Let $x_0 \in (a,b)$. If there is a constant $L \geq 0$ such that $|f^{(n+1)}(x)| \leq L$ for all $x \in (a,b)$, then*

$$R_{n+1}(x) \leq \frac{L}{(n+1)!}|x - x_0|^{n+1} = O(|x - x_0|^{n+1}) \quad \text{as} \quad x \to x_0. \tag{3.2}$$

Proof. Taylor's theorem and the assumption $|f^{(n+1)}(x)| \leq L$ for all $x \in (a,b)$ yield

$$|R_{n+1}(x)| = \left|\frac{f^{(n+1)}(c_x)}{(n+1)!}(x - x_0)^{n+1}\right| = \frac{|f^{(n+1)}(c_x)|}{(n+1)!}|x - x_0|^{n+1} \leq \frac{L}{(n+1)!}|x - x_0|^{n+1}$$

for all $x \in (a,b)$. Because $x_0 \in (a,b)$, there is $\delta > 0$ such that $(x_0 - \delta, x + \delta) \subset (a,b)$, so that the above inequality holds for all $x \in (x_0 - \delta, x + \delta)$. Therefore we can select $f(x) = R_{n+1}(x)$, $g(x) = |x - x_0|^{n+1}$ and $C = L/(n+1)!$ in Definition 2.11. This proves (3.2). □

By selecting $x = x_0 + h$ (so that $x - x_0 = h$) we can write (3.2) in the alternative form

$$R_{n+1}(x_0 + h) = O(|h|^{n+1}) \quad \text{as} \quad h \to 0$$

that shows more clearly how the remainder term tends to zero as one considers points closer and closer (i.e., $h \to 0$) to the expansion point x_0.

Example 3.5. It is important to understand that remainder term R_{n+1} can be expected to be small only in the vicinity of x_0. For example, consider the Taylor polynomial of degree $n = 2$ at the point $x_0 = 1$ for the function $f(x) = x^4 + 2x^2$. Because f is defined on the whole of $\mathbb{R}$, we can pick any interval that contains x_0 and use that in Corollary 3.4. Let us pick $[a,b] = [0,2]$. Then (as $n + 1 = 3$)

$$|f^{(3)}(x)| = \left| \frac{\mathrm{d}^2}{\mathrm{d}x^2}(4x^3 + 4x) \right| = \left| \frac{\mathrm{d}}{\mathrm{d}x}(12x^2 + 4) \right| = |24x| \leq 48 \quad \text{for all} \quad x \in [0,2].$$

Therefore we can apply Corollary 3.4 with $n = 2$, $x_0 = 1$ and $L = 48$ to conclude that

$$f(x) - P_2(x) = R_3(x) = O(|x - 1|^3) \quad \text{as} \quad x \to 1,$$

which means that the error of the degree two Taylor polynomial P_2 is *cubic* as $x \to x_0 = 1$. However, something completely different happens as one moves away from $x_0 = 1$. Because $n = 2$, we see from the definition of the Taylor polynomial in (3.1) that P_2 is a polynomial of degree two, meaning that $P_2(x) = \alpha x^2 + \beta x + \gamma$ for certain $\alpha, \beta, \gamma \in \mathbb{R}$. Since $f(x) = x^4 + 2x^2$, this means that

$$f(x) - P_2(x) = x^4 + (2 - \alpha)x^2 - \beta x - \gamma,$$

which is a polynomial of degree 4 with a positive leading coefficient (i.e., the term of degree 4 has coefficient 1). This means that

$$f(x) - P_2(x) \to \infty \quad \text{as} \quad x \to \infty \text{ or } x \to -\infty.$$

Consequently, the error can be made arbitrarily large by looking at x that is sufficiently far from $x_0 = 1$.

3.2 Linearisation

The case $n = 1$ in Taylor expansion (Definition 3.1) is one of the main tools to construct numerical methods.

Definition 3.6 (Linear approximation)**.** Let $f\colon (a,b) \to \mathbb{R}$ be a once differentiable function at a point $x_0 \in (a,b)$. The *linear approximation* to f at x_0 is the linear polynomial

$$L(x) = P_1(x) = f(x_0) + f'(x_0)(x - x_0). \tag{3.3}$$

The process of computing L and using it to approximate f is called *linearisation*.

Linear approximation is preferred over higher degree Taylor polynomials because it is so easy to compute: only $f(x_0)$ and $f'(x_0)$ are required. In applications it is often cumbersome or even impossible to obtain derivatives of orders higher than this. Moreover, linearisation

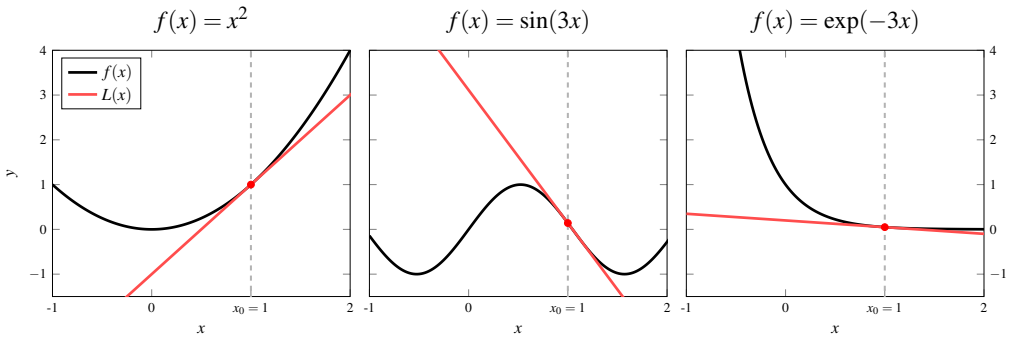


Figure 3.2: Linear approximations (red) to the three functions $f(x) = x^2$, $f(x) = \sin(3x)$ and $f(x) = \exp(-3x)$ at the point $x_0 = 1$. Observe that the linear approximations coincide with the *tangent lines* at $x_0 = 1$. That is, the linear approximation is the line whose slope coincides with the derivative of f at x_0.

easily generalises for multivariate functions $f\colon \mathbb{R}^d \to \mathbb{R}$ via the *gradient* and results in numerical methods that are straightforward to implement and understand.

Example 3.7. Let us form linear approximations to the functions

$$f_1(x) = -x^2, \quad f_2(x) = \sin(\tfrac{\pi}{2}x) \quad \text{and} \quad f_3(x) = e^x - x^2$$

at the point $x_0 = 1$. For f_1 we get

$$f_1(1) = -1 \text{ and } f_1'(1) = -2 \quad \Longrightarrow \quad L(x) = f(1) + f'(1)(x-1) = -2x + 1.$$

For f_2 we get [recall from trigonometry that $\sin(\frac{\pi}{2}) = 1$]

$$f_2(1) = 1 \text{ and } f_2'(1) = 0 \quad \Longrightarrow \quad L(x) = f(1) + f'(1)(x-1) = 1.$$

For f_3 we get

$$f_3(1) = e - 1 \text{ and } f_3'(1) = e - 2 \quad \Longrightarrow \quad L(x) = f(1) + f'(1)(x-1) = (e-2)x + 1.$$

Figure 3.2 plots linearisations at $x_0 = 1$ for three similar functions.

Example 3.7 and Figure 3.2 show that linearisation corresponds to drawing a line that (a) passes through $(x_0, f(x_0))$ and (b) whose slope equals the derivative $f'(x_0)$.

Theorem 3.8. *The linear approximation is the unique line that passes through $(x_0, f(x_0))$ and whose slope equals $f'(x_0)$.*

Proof. Every line is given by the equation $y = \alpha x + \beta$ for some $\alpha, \beta \in \mathbb{R}$, where α is the slope of the line. That a line passes through $(x_0, f(x_0))$ means that $f(x_0) = \alpha x_0 + \beta$. That the slope of the line equals $f'(x_0)$ means that $\alpha = f'(x_0)$. Therefore

$$f(x_0) = \alpha x_0 + \beta = f'(x_0)x_0 + \beta \quad \Longrightarrow \quad \beta = f(x_0) - f'(x_0)x_0.$$

Therefore any line specified in the theorem has $\alpha = f'(x_0)$ and $\beta = f(x_0) - f'(x_0)x_0$. That is, there is only one line that passes through $(x_0, f(x_0))$ and whose slope equals $f'(x_0)$. The equation

for this line is

$$y = \alpha x + \beta = f'(x_0)x + f(x_0) - f'(x_0)x_0 = f(x_0) + f'(x_0)(x - x_0),$$

which is exactly the linear approximation in (3.3). □

From Figure 3.2 it is apparent that linearisation gives an approximation that is good for x close to x_0 but that can degrade quickly in quality when one moves away from x_0. The following theorem states that the error of linear approximation has *quadratic* order as $x \to x_0$ if f is twice differentiable.

Theorem 3.9. *Suppose that $f\colon [a,b] \to \mathbb{R}$ is continuous on $[a,b]$ and twice differentiable on (a,b). Let $x_0 \in (a,b)$. If there is a constant $L \geq 0$ such that $|f''(x)| \leq L$ for all $x \in (a,b)$, then*

$$f(x) - L(x) = O(|x - x_0|^2) \quad \text{as} \quad x \to x_0.$$

Proof. Because $L(x) = P_1(x)$, the claim follows by setting $n = 1$ in Corollary 3.4. □

3.3 Array programming

Array programming, or *vectorisation*, refers to a style of programming where mathematical operations are applied to an array of values at once, rather than to each value individually (i.e., via a loop). Array programming tends to be significantly more *concise* and *easier to read* than an equivalent construction based on loops. In many high-level programming languages, such as Python and MATLAB, array programming is also *significantly faster* than loops and should therefore be used whenever possible. Suppose that we want to square (i.e., compute x^2) for ten million x between 0 and 1 with Python. For array programming we use the Python library `numpy`.

```
>>> import numpy as np
>>> import time
>>> # an array of ten million numbers between 0 and 1
>>> a = np.linspace(start=0, stop=1, num=int(1e7))
>>> # use loop to square each number
>>> start1 = time.perf_counter()
>>> b1 = [x**2 for x in a]
>>> end1 = time.perf_counter()
>>> # square each number using array programming
>>> start2 = time.perf_counter()
>>> b2 = a**2
>>> end2 = time.perf_counter()
>>> # print elapsed times
>>> print(end1 - start1)
0.5299568176269531
>>> print(end2 - start2)
0.007016181945800781
```

The elapsed times depend on hardware and vary slightly from one run to the next. But what remains constant is that the second approach based on array programming is much faster (in this case by approximately two orders of magnitude).

Example 3.10. Monte Carlo integration is a straightforward technique to approximate the integral of a function. Suppose that we want to approximate the integral $\int_0^1 f(x)\,\mathrm{d}x$ for some integrand function $f\colon [0,1] \to \mathbb{R}$. In Monte Carlo integration we draw n numbers $x_1, \ldots, x_n$ randomly on $[0,1]$ and approximate the integral as the average of integrand evaluations at

these points:

$$\int_0^1 f(x)\,\mathrm{d}x \approx \frac{1}{n}\sum_{i=1}^{n} f(x_i). \tag{3.4}$$

In programming languages that support array programming, no loops are needed to compute the Monte Carlo approximation in (3.4).

In Python the Monte Carlo approximation ($n = 200$) to the integral of $f(x) = x^{3.4} + \sin(x)$ could be computed as follows:

```
>>> import numpy as np
>>> rng = np.random.default_rng()
>>> # draw n = 200 random numbers on [0, 1]
>>> n = 200
>>> samples = rng.random(size=n)
>>> # evaluate f at these point and compute average
>>> evals = np.power(samples, 3.4) + np.sin(samples)
>>> mc_approx = np.sum(evals) / n
>>> print(mc_approx)
0.692222065735476
```

The final result will vary from one run to the next because the 200 samples are random. In MATLAB this could be done as follows:

```
>>> # draw n = 200 random numbers on [0, 1]
>>> n = 200;
>>> samples = rand(n, 1);
>>> # evaluate f at these point and compute average
>>> evals = samples.^(3.4) + sin(samples);
>>> mc_approx = sum(evals) / n
mc_approx =

    0.6686
```

Here we used MATLAB's elementwise power signified by dot to compute $x^{3.4}$ for every sample x.

4 Numerical Integration

Recall that the *antiderivative* (or *indefinite integral*) of a function f is a function F such that

$$F'(x) = f(x).$$

Let $a < b$. The *definite integral* of f from a to b represents the area between the graph of f and the x-axis. The definite integral can be expressed in terms of the antiderivative as

$$\int_a^b f(x)\,\mathrm{d}x = F(b) - F(a)$$

by the fundamental theorem of calculus. Differentiation is easy. For example, if one is familiar with the chain rule and knows how to differentiate exponentials, polynomials and trigonometric functions, there is no difficulty in differentiating the function $f(x) = \exp(-x^2 + \sin(x))$. In contrast, integration tends to be difficult or outright impossible. The antiderivative of $f(x) = \exp(-x^2 + \sin(x))$ does not admit a closed form expression in terms of the familiar elementary functions[2]. As a result, numerical methods are necessary to compute most definite integrals. To compute an antiderivative one must use *symbolic computation* that is not discussed on this course. Note that a function whose antiderivative is not available in terms of elementary functions can nevertheless have a simple definite integral. For example,

$$\int_{-\infty}^{\infty} \exp(-x^2)\,\mathrm{d}x = \sqrt{\pi}.$$

It is useful to think of definite integrals as areas and *numerical integration* methods as methods that estimate areas. Such methods are called quadrature rules (or simply rules). The terminology comes from the Latin word *quadratus* for "square" (particularly apt for the midpoint rule of Section 4.1).

Definition 4.1 (Quadrature rule)**.** Let $n \in \mathbb{N}$. A *quadrature rule* with *weights* $w_1, \ldots, w_n \in \mathbb{R}$ and *nodes* $t_1, \ldots, t_n \in [a,b]$ is the following integral approximation:

$$\int_a^b f(x)\,\mathrm{d}x \approx \sum_{i=1}^n w_i f(t_i). \tag{4.1}$$

How the weights and nodes are selected determines how good a quadrature rule is. This chapter discusses three quadrature rules, the midpoint rule, the trapezoidal rule, and Simpson's rule.

4.1 Midpoint Rule

Let $x_i = a + (b-a)i/n$ for $i \in \{0, 1, \ldots, n\}$ be equispaced points $[a,b]$. Recall from the course on integral calculus that the integral of a sufficiently regular (i.e., Riemann integrable) function is obtained as the limit of a Riemann sum:

$$\int_a^b f(x)\,\mathrm{d}x = \lim_{n\to\infty} \sum_{i=1}^n (x_i - x_{i-1}) f(t_i) = \lim_{n\to\infty} \frac{b-a}{n} \sum_{i=1}^n f(t_i), \tag{4.2}$$

where $t_i \in [x_{i-1}, x_i]$. Each term in the sums represents *the area of a rectangle* with base width $(x_i - x_{i-1}) = (b-a)/n$ and height $f(t_i)$. The heights of the rectangles can be determined using

[2] A function is typically considered *elementary* if it is composed out of finitely many constants, polynomials, exp, log, trigonometric and hyperbolic functions (sin, cos, tan, sinh, cosh, tanh and so on) and inverses of trigonometric and hyperbolic functions.

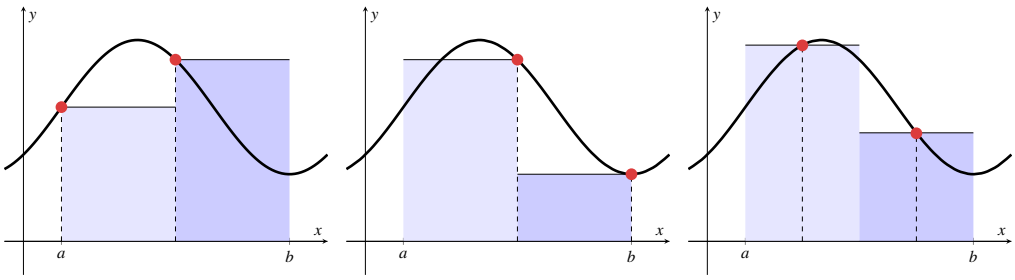


Figure 4.1: Approximations of the definite integral $\int_a^b f(x)\,\mathrm{d}x$ using two rectangles whose heights are calculated at the starting point (*left*), at the endpoint (*middle*), and at the midpoint (*right*) of each subinterval. The midpoint rule in Definition 4.2 corresponds to the rightmost figure.

any value of the function in the subintervals $[x_{i-1}, x_i]$. The simplest approach is to calculate the height at an endpoint of the interval, which translates to replacing $f(t_i)$ with $f(x_{i-1})$ or $f(x_i)$ in (4.2). By replacing $f(t_i)$ with $f(\frac{1}{2}(x_i + x_{i-1}))$, where $\frac{1}{2}(x_i + x_{i-1})$ is the *midpoint* of the subinterval $[x_{i-1}, x_i]$, and the limit in (4.2) with a finite n yields the midpoint rule, which is conceptually similar to the difference quotient approximation from Example 2.5 in that an approximation is constructed by "truncating a limiting process". The three approaches are illustrated in Figure 4.1.

Definition 4.2 (Midpoint rule)**.** Let $x_i = a + (b-a)i/n$ for $i \in \{0, 1, \ldots, n\}$. The *midpoint rule* (or *rectangle rule*) is the following integral approximation:

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{b-a}{n} \sum_{i=1}^{n} f\left(\frac{x_i + x_{i-1}}{2}\right) = h \sum_{i=1}^{n} f\left(\frac{x_i + x_{i-1}}{2}\right), \tag{4.3}$$

where $h = x_{i-1} - x_i = (b-a)/n$ is the length of each subinterval $[x_{i-1}, x_i]$.

By comparing (4.3) to (4.1) we see that the midpoint rule is a quadrature rule with the nodes $t_i = \frac{1}{2}(x_i + x_{i-1})$ and uniform weights $w_i = (b-a)/n$. Figure 4.2 contains a graphical illustration of the midpoint rule with two (i.e., $n = 2$) subintervals.

Remark 4.3. We could have used non-equispaced points $a = x_0 < x_1 < \cdots < x_{n-1} < x_n = b$ to construct the midpoint rule. The resulting rule is

$$\int_a^b f(x)\,\mathrm{d}x \approx \sum_{i=1}^{n} (x_i - x_{i-1}) f\left(\frac{x_i + x_{i-1}}{2}\right).$$

All quadrature rules defined in this section can be constructed for non-equispaced points. However, for simplicity we always use equispaced points.

Example 4.4. Let us use the midpoint rule with four subintervals (i.e., $n = 4$) to approximate the integral $\int_0^4 x^3\,\mathrm{d}x = 4^4/4 = 64$. To compute the right-hand side of (4.3) we need (a) the length of the subintervals, $h = (b-a)/n$, and (b) the midpoints $\frac{1}{2}(x_i + x_{i-1})$. Because $n = 4$, $b = 4$ and $a = 0$, we have

$$\frac{b-a}{n} = \frac{4-0}{4} = 1 \tag{4.4}$$

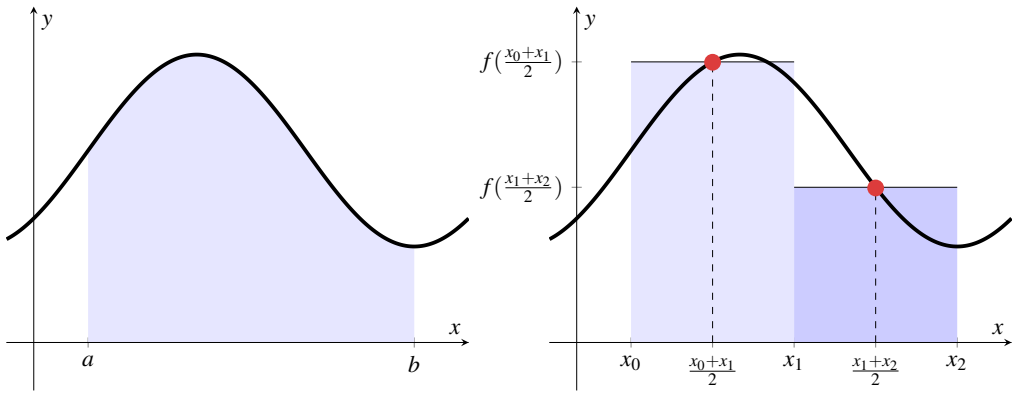


Figure 4.2: *Left*: The area corresponding to the definite integral $\int_a^b f(x)\,\mathrm{d}x$. *Right*: Its approximation using the midpoint rule with two (i.e., $n = 2$) subintervals.

and

$$x_i = a + \frac{(b-a)i}{n} = 0 + \frac{(4-0)i}{4} = i.$$

Therefore the midpoints are

$$\frac{x_i + x_{i-1}}{2} = \frac{i + (i-1)}{2} = i - \frac{1}{2}. \tag{4.5}$$

Inserting (4.4) and (4.5) in (4.3) gives

$$\begin{aligned}\int_0^4 x^3\,\mathrm{d}x \approx 1 \cdot \sum_{i=1}^{4} f\left(i - \frac{1}{2}\right) &= 1 \cdot f\left(\frac{1}{2}\right) + 1 \cdot f\left(\frac{3}{2}\right) + 1 \cdot f\left(\frac{5}{2}\right) + 1 \cdot f\left(\frac{7}{2}\right) \\ &= \left(\frac{1}{2}\right)^3 + \left(\frac{3}{2}\right)^3 + \left(\frac{5}{2}\right)^3 + \left(\frac{7}{2}\right)^3 \\ &= 62.\end{aligned}$$

Therefore the relative error (recall Definition 2.8) of the midpoint rule is

$$\left|\frac{\int_0^4 x^3\,\mathrm{d}x - 62}{\int_0^4 x^3\,\mathrm{d}x}\right| = \left|\frac{64-62}{64}\right| = \frac{1}{32} = 3.125\ \%.$$

A quadrature rule is said to be *exact* for a function f if the integral approximation coincides with the true integral.

Theorem 4.5. *The midpoint rule is exact for every polynomial of degree at most one.*

Proof. Every polynomial of degree at most one takes the form $f(x) = \alpha + \beta x$ for some constants $\alpha, \beta \in \mathbb{R}$. We want to show that

$$\int_a^b f(x)\,\mathrm{d}x = \frac{b-a}{n} \sum_{i=1}^{n} f\left(\frac{x_i + x_{i-1}}{2}\right)$$

for such f. The integral is

$$\int_a^b f(x)\,\mathrm{d}x = \int_a^b (\alpha + \beta x)\,\mathrm{d}x = (b-a)\alpha + \frac{(b^2 - a^2)\beta}{2}. \tag{4.6}$$

Because $x_i = a + (b-a)i/n$, the midpoints are

$$\frac{x_i + x_{i-1}}{2} = a + \frac{(b-a)i/n + (b-a)(i-1)/n}{2} = a + \frac{(b-a)i}{n} - \frac{b-a}{2n}.$$

Therefore the midpoint rule is

$$\begin{aligned}\frac{b-a}{n}\sum_{i=1}^{n} f\left(\frac{x_i + x_{i-1}}{2}\right) &= \frac{b-a}{n}\left(\alpha n + \beta \sum_{i=1}^{n}\left[a + \frac{(b-a)i}{n} - \frac{b-a}{2n}\right]\right) \\ &= (b-a)\alpha + \frac{b-a}{n}\beta\left(na - \frac{b-a}{2} + \frac{b-a}{n}\sum_{i=1}^{n} i\right) \\ &= (b-a)\alpha + (b-a)a\beta - \frac{(b-a)^2\beta}{2n} + \frac{(b-a)^2\beta}{n^2}\sum_{i=1}^{n} i.\end{aligned}$$

Recall then the arithmetic sum formula $\sum_{i=1}^{n} i = n(n+1)/2$, which yields

$$\begin{aligned}\frac{b-a}{n}\sum_{i=1}^{n} f\left(\frac{x_i + x_{i-1}}{2}\right) &= (b-a)\alpha + \frac{(b-a)a\beta}{n} - \frac{(b-a)^2\beta}{2n} + \frac{(b-a)^2\beta}{n^2}\cdot\frac{n(n+1)}{2} \\ &= (b-a)\alpha + (b-a)a\beta - \frac{(b-a)^2\beta}{2n} + (n+1)\frac{(b-a)^2\beta}{2n} \\ &= (b-a)\alpha + (b-a)a\beta + \frac{(b-a)^2\beta}{2} \\ &= (b-a)\alpha + (b-a)a\beta + \frac{(b^2 - 2ab + a^2)\beta}{2} \\ &= (b-a)\alpha + \frac{(2(b-a)a + b^2 - 2ab + a^2)\beta}{2} \\ &= (b-a)\alpha + \frac{(b^2 - a^2)\beta}{2}.\end{aligned}$$

This equals (4.6). Thus the midpoint rule is exact for all functions of the form $f(x) = \alpha + \beta x$. □

The midpoint rule is *not* exact for all polynomials of degree two. To show this it suffices to come up with one polynomial of degree two for which (4.3) is not an equality. Select $f(x) = x^2$ and consider the case $a = 0$ for simplicity. Then

$$\int_a^b f(x)\,\mathrm{d}x = \int_0^b x^2\,\mathrm{d}x = \frac{b^3}{3} \tag{4.7}$$

and (as in the proof of Theorem 4.5)

$$\begin{aligned}\frac{b-a}{n}\sum_{i=1}^{n} f\left(\frac{x_i + x_{i-1}}{2}\right) = \frac{b-a}{n}\sum_{i=1}^{n}\left(a + \frac{(b-a)i}{n} - \frac{b-a}{2n}\right)^2 &= \frac{b}{n}\sum_{i=1}^{n}\left(\frac{bi}{n} - \frac{b}{2n}\right)^2 \\ &= \frac{b^3}{n^3}\sum_{i=1}^{n}\left(i - \frac{1}{2}\right)^2 \\ &= \frac{b^3}{n^3}\sum_{i=1}^{n}\left(i^2 - i + \frac{1}{4}\right).\end{aligned}$$

Recall the summation formulae

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \quad \text{and} \quad \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6},$$

which yield

$$\begin{aligned}\frac{b^3}{n^3}\sum_{i=1}^{n}\left(i^2-i+\frac{1}{4}\right) &= \frac{b^3}{n^3}\left(\frac{n(n+1)(2n+1)}{6}-\frac{n(n+1)}{2}+\frac{n}{4}\right)\\ &= \frac{b^3}{n^3}\cdot\frac{n}{12}[2n(n+1)(2n+1)-6n(n+1)+3n]\\ &= \frac{b^3(4n^3+6n^2+2n-6n^2-6n+3n)}{12n^2}\\ &= \frac{b^3(4n^2-1)}{12n^2}.\end{aligned}$$

Therefore

$$\frac{b-a}{n}\sum_{i=1}^{n} f\left(\frac{x_i+x_{i-1}}{2}\right) = \frac{b^3}{3}\left(1-\frac{1}{4n^2}\right),$$

which is not equal to the true integral in (4.7) for any $n \in \mathbb{N}$ because $1-1/(4n^2) \neq 1$.

Theorem 4.6. *Suppose that $f\colon [a,b] \to \mathbb{R}$ is continuous on $[a,b]$ and twice differentiable on (a,b). If there is a constant $L \geq 0$ such that $|f''(x)| \leq L$ for all $x \in (a,b)$, then the midpoint rule satisfies*

$$\left|\int_a^b f(x)\,\mathrm{d}x - \sum_{i=1}^{n} h\cdot f\left(\frac{x_i+x_{i-1}}{2}\right)\right| \leq \frac{(b-a)^3L}{24n^2} = \frac{(b-a)L}{24}h^2,$$

where $h = x_i - x_{i-1} = (b-a)/n$ is the length of each subinterval $[x_{i-1}, x_i]$.

Proof. By breaking the integration interval into n subintervals we may write the error of the midpoint rule as

$$\int_a^b f(x)\,\mathrm{d}x - \sum_{i=1}^{n} h\cdot f\left(\frac{x_i+x_{i-1}}{2}\right) = \sum_{i=1}^{n}\left(\int_{x_{i-1}}^{x_i} f(x)\,\mathrm{d}x - h\cdot f\left(\frac{x_i+x_{i-1}}{2}\right)\right). \tag{4.8}$$

Let $t_i = \frac{1}{2}(x_i+x_{i-1})$ denote the ith midpoint. We have

$$\begin{aligned}\int_{x_{i-1}}^{x_i}[f(t_i)+f'(t_i)(x-t_i)]\,\mathrm{d}x &= f(t_i)(x_i-x_{i-1}) + \frac{f'(t_i)}{2}(x_i^2-x_{i-1}^2) - f'(t_i)t_i(x_i-x_{i-1})\\ &= (x_i-x_{i-1})\left[f(t_i)+\frac{f'(t_i)}{2}(x_i+x_{i-1}) - f'(t_i)t_i\right]\\ &= (x_i-x_{i-1})[f(t_i)-f'(t_i)t_i+f'(t_i)t_i]\\ &= (x_i-x_{i-1})f(t_i)\\ &= h\cdot f\left(\frac{x_i+x_{i-1}}{2}\right).\end{aligned}$$

Therefore (4.8) becomes

$$\int_a^b f(x)\,\mathrm{d}x - \sum_{i=1}^{n} h\cdot f\left(\frac{x_i+x_{i-1}}{2}\right) = \sum_{i=1}^{n}\int_{x_{i-1}}^{x_i}[f(x)-f(t_i)-f'(t_i)(x-t_i)]\,\mathrm{d}x.$$

Observe that $f(x)-f(t_i)-f'(t_i)(x-t_i) = f(x)-P_{1,t_i}(x) = R_{2,t_i}(x)$, where P_{1,t_i} and R_{2,t_i} are the Taylor polynomial of degree one and its remainder term from Section 3.1 with $x_0 = t_i$. From Taylor's theorem (Theorem 3.3) and Corollary 3.4 with $n+1=2$ it then follows that

$$\begin{aligned}\left|\int_a^b f(x)\,\mathrm{d}x - \sum_{i=1}^{n} h\cdot f\left(\frac{x_i+x_{i-1}}{2}\right)\right| = \left|\sum_{i=1}^{n}\int_{x_{i-1}}^{x_i} R_{2,t_i}(x)\,\mathrm{d}x\right| &\leq \sum_{i=1}^{n}\int_{x_{i-1}}^{x_i}\frac{L}{2!}(x-t_i)^2\,\mathrm{d}x\\ &= \frac{L}{2}\sum_{i=1}^{n}\int_{-h/2}^{h/2} y^2\,\mathrm{d}y,\end{aligned}$$

where the last equation used the change of variables $y = x - t_i$ and the fact that t_i is the midpoint of $[x_{i-1}, x_i]$. Therefore

$$\begin{aligned}\left|\int_a^b f(x)\,\mathrm{d}x - \sum_{i=1}^n h \cdot f\left(\frac{x_i + x_{i-1}}{2}\right)\right| = \frac{L}{2}\sum_{i=1}^n \int_{-h/2}^{h/2} y^2\,\mathrm{d}y &= \frac{L}{2}\sum_{i=1}^n \frac{1}{3}\cdot[(h/2)^3 - (-h/2)^3] \\ &= \frac{L}{24}\sum_{i=1}^n h^3 \\ &= \frac{L}{24} n h^3 \\ &= \frac{L}{24} n \left(\frac{b-a}{n}\right)^3 \\ &= \frac{(b-a)^3 L}{24 n^2}. \end{aligned}$$

□

With big-O notation from Section 2.4 we can express Theorem 4.6 as

$$\left|\int_a^b f(x)\,\mathrm{d}x - \frac{b-a}{n}\sum_{i=1}^n f\left(\frac{x_i + x_{i-1}}{2}\right)\right| = O(n^{-2}) \quad \text{as} \quad n \to \infty.$$

This gives information about the order of the integration error as n, the number of subintervals grows. Equivalently, we can write

$$\left|\int_a^b f(x)\,\mathrm{d}x - h\sum_{i=1}^n f\left(\frac{x_i + x_{i-1}}{2}\right)\right| = O(h^2) \quad \text{as} \quad h \to 0,$$

which describes the error as $h = (b-a)/n$, the length of each subinterval, tends to zero.

4.2 Trapezoidal Rule

The midpoint rule used rectangles to approximate an integral. The trapezoidal rule uses *trapezoids*, which generalise rectangles. Both pairs of sides of a rectangle are parallel; a trapezoid has at least one pair of parallel sides. Whereas a rectangle with base length h and height a has area $a \cdot h$, a trapezoid with base length h and heights b_1 and b_1 has area $\frac{1}{2}(b_1 + b_2)\cdot h$ (see Figure 4.3). By replacing rectangles whose heights are computed from the value of f at the midpoints of the subintervals, $\frac{1}{2}(x_i + x_{i-1})$, with trapezoids whose side heights are given by the values of f at the endpoints of the subintervals, x_{i-1} and x_i, we obtain the trapezoidal rule.

Definition 4.7 (Trapezoidal rule)**.** Let $x_i = a + (b-a)i/n$ for $i \in \{0, 1, \ldots, n\}$. The *trapezoidal rule* is the following integral approximation:

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{b-a}{n}\sum_{i=1}^n \frac{f(x_i) + f(x_{i-1})}{2} = h\sum_{i=1}^n \frac{f(x_i) + f(x_{i-1})}{2}, \tag{4.9}$$

where $h = x_i - x_{i-1} = (b-a)/n$ is the length of each subinterval $[x_{i-1}, x_i]$.

By noting that $f(x_0)$ and $f(x_n)$ appear once and all other function evaluations twice in (4.9), we can write the trapezoidal rule as

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{h}{2}[f(x_0) + 2f(x_1) + \cdots + 2f(x_{n-1}) + f(x_n)]. \tag{4.10}$$

Therefore the trapezoidal rule is a quadrature rule [recall (4.1)] with nodes $t_i = x_i$ for $i \in \{0, \ldots, n\}$ and weights $w_0 = w_n = \frac{1}{2}h$ and $w_1 = \cdots = w_{n-1} = h$. The trapezoidal rule is illustrated in Figure 4.4.

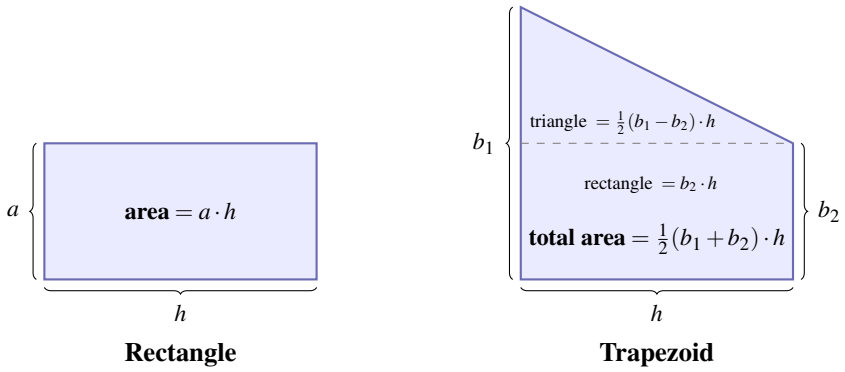


Figure 4.3: Areas of a rectangle and a trapezoid with two right angles.

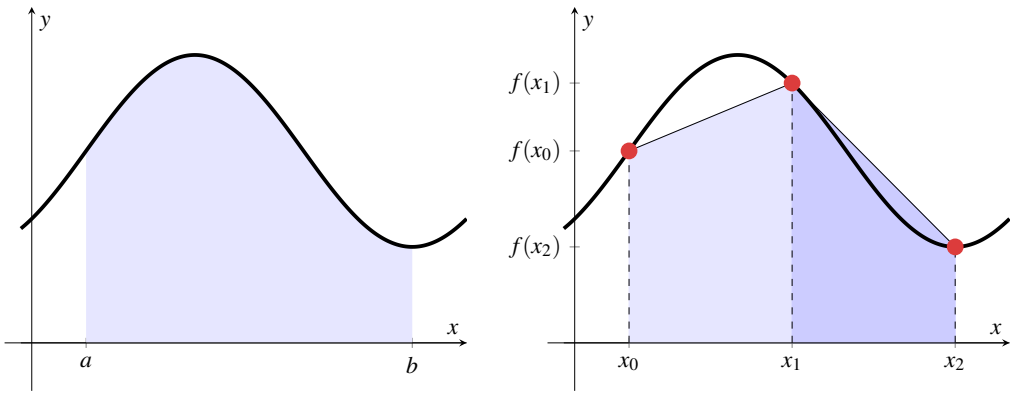


Figure 4.4: *Left*: The area corresponding to the definite integral $\int_a^b f(x)\,\mathrm{d}x$. *Right*: Its approximation using the trapezoidal rule with two (i.e., $n = 2$) subintervals.

Example 4.8. Let us use the trapezoidal rule with four subintervals (i.e., $n = 4$) to approximate the integral $\int_0^4 x^3\,\mathrm{d}x = 4^4/4 = 64$. To compute the right-hand side of (4.9) we need (a) the length of the subintervals, $h = (b-a)/n$, and (b) the points x_i. Because $n = 4$, $b = 4$ and $a = 0$, we have

$$h = \frac{b-a}{n} = \frac{4-0}{4} = 1 \tag{4.11}$$

and

$$x_i = a + \frac{(b-a)i}{n} = 0 + \frac{(4-0)i}{4} = i. \tag{4.12}$$

Inserting (4.11) and (4.12) in (4.10) gives

$$\begin{aligned}\int_0^4 x^3\,\mathrm{d}x &\approx \frac{h}{2}[f(x_0) + 2f(x_1) + 2f(x_2) + 2f(x_3) + f(x_4)] \\ &= \frac{1}{2}[f(0) + 2f(1) + 2f(2) + 2f(3) + f(4)] \\ &= \frac{1}{2}[0^3 + 2\cdot 1^3 + 2\cdot 2^3 + 2\cdot 3^3 + 4^3] \\ &= 68.\end{aligned}$$

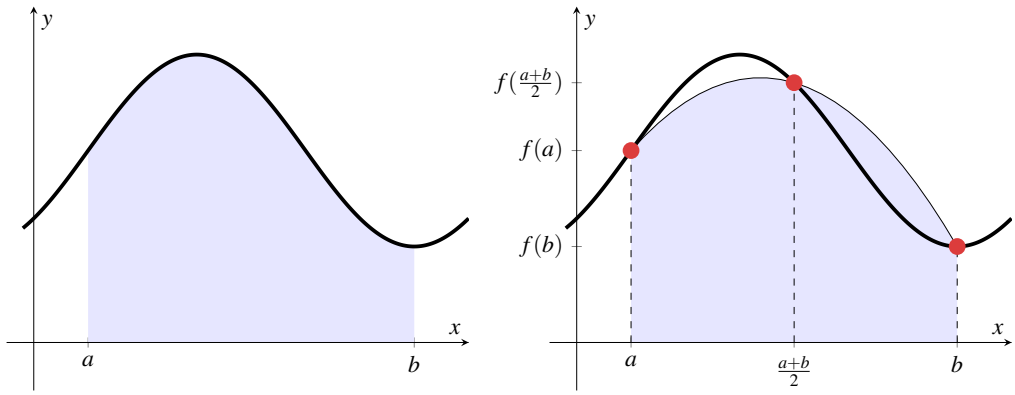


Figure 4.5: *Left*: The area corresponding to the definite integral $\int_a^b f(x)\,\mathrm{d}x$. *Right*: Its approximation using Simpson's rule in (4.14) with three points.

Therefore the relative error (recall Definition 2.8) of the trapezoidal rule is

$$\left|\frac{\int_0^4 x^3\,\mathrm{d}x - 68}{\int_0^4 x^3\,\mathrm{d}x}\right| = \left|\frac{64-68}{64}\right| = \frac{1}{16} = 6.25\ \%.$$

The trapezoidal rule behaves in a manner similar to the midpoint rule. You should compare the following theorems to Theorems 4.5 and 4.6.

Theorem 4.9. *The trapezoidal rule is exact for all polynomials of degree at most one.*

Proof. Similar to the proof of Theorem 4.5 and is left as an exercise. □

Theorem 4.10. *Suppose that $f\colon [a,b] \to \mathbb{R}$ is continuous on $[a,b]$ and twice differentiable on (a,b). If there is a constant $L \geq 0$ such that $|f''(x)| \leq L$ for all $x \in (a,b)$, then the trapezoidal rule satisfies*

$$\left|\int_a^b f(x)\,\mathrm{d}x - \sum_{i=1}^n h \cdot \frac{f(x_i)+f(x_{i-1})}{2}\right| \leq \frac{(b-a)^3 L}{12n^2} = \frac{(b-a)L}{12}h^2,$$

where $h = x_i - x_{i-1} = (b-a)/n$ is the length of each subinterval $[x_{i-1}, x_i]$.

Proof. We omit the proof, which is similar to that of Theorem 4.6. □

4.3 Simpson's Rule

The midpoint rule uses rectangles and the trapezoidal rule trapezoids to approximate an integral. The midpoint rule is constructed using a single value of f on each subinterval $[x_{i-1}, x_i]$ (i.e., the value of f at the midpoint) while the trapezoidal rule uses two values (the values of f at the endpoints). Simpson's rule uses three values on each subinterval: the endpoints and the midpoint. Three values of f at given points determine a unique quadratic polynomial (i.e., a quadratic *interpolant*) that coincides with f at those points and whose integral (i.e., the area under the graph of the polynomial) can be used to approximate the integral of f.

Suppose that we have only one subinterval, which is simply $[a,b]$. The midpoint is $(a+b)/2$. The quadratic polynomial $\alpha+\beta x+\gamma x^2$ that coincides with f at the endpoints a and b and at the midpoint $c=(a+b)/2$ must satisfy the three equations

$$\begin{aligned}\alpha+\beta c+\gamma c^2 &= f(c),\\ \alpha+\beta a+\gamma a^2 &= f(a),\\ \alpha+\beta b+\gamma b^2 &= f(b).\end{aligned} \tag{4.13}$$

Because we have three equations for three unknowns (α, β and γ), it is straightforward to solve (the computations are omitted) the unique coefficients for which these three equations hold. Because

$$\int_a^b (\alpha+\beta x+\gamma x^2)\,\mathrm{d}x = (b-a)\alpha+\frac{(b^2-a^2)\beta}{2}+\frac{(b^3-a^3)\gamma}{3}$$

for any $\alpha,\beta,\gamma\in\mathbb{R}$, we can use the coefficients just computed to obtain the integral approximation

$$\int_a^b f(x)\,\mathrm{d}x \approx (b-a)\alpha+\frac{(b^2-a^2)\beta}{2}+\frac{(b^3-a^3)\gamma}{3}.$$

For the coefficients that solve the equations in (4.13) the right-hand side ends up simplifying considerably, so that we finally obtain

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{b-a}{6}\left[f(a)+4f\left(\frac{a+b}{2}\right)+f(b)\right]. \tag{4.14}$$

This approximation is illustrated in Figure 4.5. To construct an approximation that uses more than three evaluations of f we can stitch together multiple approximations of this type. Namely, let n be even and $x_i=a+hi$ for $h=(b-a)/n$. Then

$$\int_a^b f(x)\,\mathrm{d}x = \sum_{k=1}^{\frac{n}{2}}\int_{x_{2k-2}}^{x_{2k}} f(x)\,\mathrm{d}x \approx \sum_{k=1}^{\frac{n}{2}}\frac{x_{2k}-x_{2k-2}}{6}[f(x_{2k-2})+4f(x_{2k-1})+f(x_{2k})],$$

where we applied (4.14) with $a=x_{2k-2}$ and $b=x_{2k}$ to each of the integrals and used the fact that in this case $\frac{1}{2}(a+b)=x_{2k-1}$ (i.e., now *two* subintervals are used for *one* quadratic interpolation). Grouping terms corresponding to odd and even indices gives the full Simpson's rule.

Definition 4.11 (Simpson's rule)**.** Let $n\in\mathbb{N}$ be even and $x_i=a+(b-a)i/n$ for $i\in\{0,1,\ldots,n\}$. *Simpson's rule* is the following integral approximation:

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{h}{3}[f(x_0)+4f(x_1)+2f(x_2)+\cdots+2f(x_{n-2})+4f(x_{n-1})+f(x_n)], \tag{4.15}$$

where $h=x_i-x_{i-1}=(b-a)/n$ is the length of each subinterval $[x_{i-1},x_i]$.

We can use summation notation to express Simpson's rule as

$$\int_a^b f(x)\,\mathrm{d}x = \frac{h}{3}\left(f(x_0)+f(x_n)+4\sum_{i=1}^{\frac{n}{2}} f(x_{2i-1})+2\sum_{i=1}^{\frac{n}{2}-1} f(x_{2i})\right).$$

From this we see that the endpoints of the interval $[a,b]$ receive weights $\frac{1}{3}h$, odd points (x_{2i-1}) receive weights $\frac{4}{3}h$ and even points (x_{2i}) receive weights $\frac{2}{3}h$.

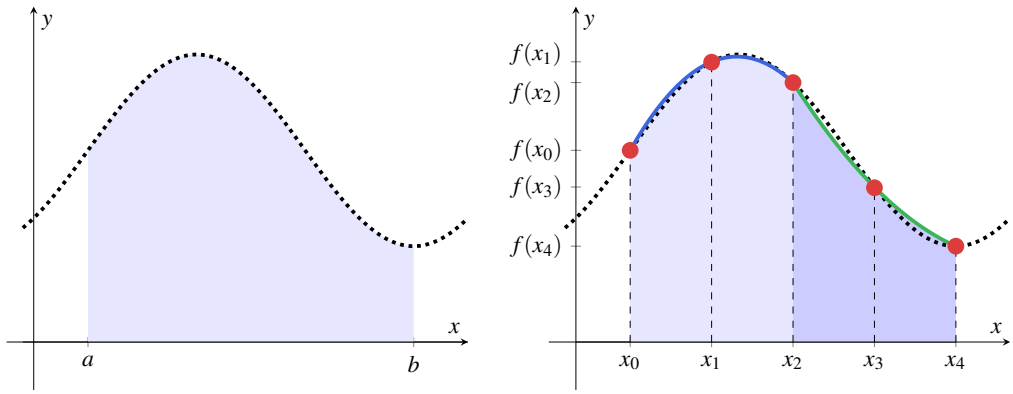


Figure 4.6: *Left*: The area corresponding to the definite integral $\int_a^b f(x)\,\mathrm{d}x$. *Right*: Its approximation using Simpson's rule in (4.12) with $n = 4$. Note that one quadratic polynomial is fitted on the subintervals $[x_0, x_1] \cup [x_1, x_2] = [x_0, x_2]$ and another on the subintervals $[x_2, x_3] \cup [x_3, x_4] = [x_2, x_4]$.

Example 4.12. Let us use Simpson's rule with $n = 4$ to approximate the integral $\int_0^4 x^3\,\mathrm{d}x = 4^4/4 = 64$. To compute the right-hand side of (4.9) we need (a) the length of the subintervals, $h = (b-a)/n$, and (b) the points x_i. Because $n = 4$, $b = 4$ and $a = 0$, we have

$$h = \frac{b-a}{n} = \frac{4-0}{4} = 1 \tag{4.16}$$

and

$$x_i = a + \frac{(b-a)i}{n} = 0 + \frac{(4-0)i}{4} = i. \tag{4.17}$$

Inserting (4.16) and (4.17) in (4.12) gives

$$\begin{aligned}\int_0^4 x^3\,\mathrm{d}x &\approx \frac{h}{3}[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 4f(x_{n-1}) + f(x_n)] \\ &= \frac{1}{3}[f(0) + 4f(1) + 2f(2) + 4f(3) + f(1)] \\ &= \frac{1}{3}[0^3 + 4\cdot 1^3 + 2\cdot 2^3 + 4\cdot 3^3 + 4^3] \\ &= 64.\end{aligned}$$

Therefore the relative error (recall Definition 2.8) of Simpson's rule is

$$\left|\frac{\int_0^4 x^3\,\mathrm{d}x - 64}{\int_0^4 x^3\,\mathrm{d}x}\right| = \left|\frac{64-64}{64}\right| = 0 = 0\,\%.$$

It is no coincidence that Simpson's rule was exact in Example (4.12).

Theorem 4.13. *Simpson's rule is exact for all polynomials of degree at most three.*

Proof. Similar to the proof of Theorem 4.5 but more complicated. □

Let

$$E_n(f) = \left|\int_a^b f(x)\,\mathrm{d}x - \frac{h}{3}\left(f(x_0) + f(x_n) + 4\sum_{i=1}^{\frac{n}{2}} f(x_{2i-1}) + 2\sum_{i=1}^{\frac{n}{2}-1} f(x_{2i})\right)\right|$$

denote the absolute error of Simpson's rule.

Theorem 4.14. *Suppose that $f\colon [a,b] \to \mathbb{R}$ is continuous on $[a,b]$ and four times differentiable on (a,b). If there is a constant $L \geq 0$ such that $|f^{(4)}(x)| \leq L$ for all $x \in (a,b)$, then Simpson's rule satisfies*

$$E_n(f) \leq \frac{(b-a)^5 L}{180 n^4} = \frac{(b-a)L}{180} h^4,$$

where $h = x_i - x_{i-1} = (b-a)/n$ is the length of each subinterval $[x_{i-1}, x_i]$.

Proof. Omitted. □

If f is sufficiently smooth (i.e., has sufficiently many derivatives), Theorems 4.6, 4.10 and 4.14 show that the errors of the midpoint and trapezoidal rules are of order h^2 while the error of Simpson's rule is of order h^4. Therefore Simpson's rule may converge faster than the midpoint or trapezoidal rule if f is sufficiently smooth.

5 Solutions to Systems of Equations

Problems encountered while studying mathematics typically admit precise closed-form solutions. This might give the impression that every equation can be solved exactly. However, the opposite is true: most equations cannot be solved in closed form in terms of a finite number of elementary functions. This is almost always the case when the equation involves different types of elementary functions. For example, the solution to

$$e^x = 2$$

is $x = \log 2 \approx 0.693$. But the solution to

$$e^x + x = 2$$

is $x = 2 - W(e^2) \approx 0.442$, where W is a special function called the *Lambert W function*. Finally, the solution to

$$e^x + x + \sin(x) = 2,$$

which is $x \approx 0.316$, lacks an expression in terms of any standard elementary or special functions. Solutions to equations that cannot be solved analytically are obtained using *numerical root-finding methods*.

If f and g are real-valued functions of one real variable, an $x^* \in \mathbb{R}$ is *a solution to the equation*

$$f(x) = g(x)$$

if $f(x^*) = g(^*)$. Similarly, if $\mathbf{f}, \mathbf{g} \colon \mathbb{R}^m \to \mathbb{R}^n$, an $\mathbf{x}^* \in \mathbb{R}^m$ is a solution to

$$\mathbf{f}(\mathbf{x}) = \mathbf{g}(\mathbf{x})$$

if $\mathbf{f}(\mathbf{x}^*) = \mathbf{g}(\mathbf{x}^*)$. A solution to an equation is said to *solve the equation*.

Example 5.1. Consider the equation $x^2 + 1 = 2$, so that $f(x) =^2 +1$ and $g(x) = 2$. The real $x = -1$ is a solution to the equation $x^2 + 1 = 2$ because

$$f(-1) = (-1)^2 + 1 = 1 + 1 = 2 = g(-1),$$

but $x = 2$ is not a solution because

$$f(2) = 2^2 + 1 = 4 + 1 = 5 \neq 2 = g(2).$$

Graphically the solutions of an equation correspond to the points on the x-axis where the graphs of the functions f and g in the equation $f(x) = g(x)$ intersect. For example, the solutions of the equation $x^2 + 1 = 2$ are $x_0 = \pm 1$ because at these points the graphs of the functions $f(x) = x^2 + 1$ and $g(x) = 2$ are at the same height. The functions f and g can also be chosen differently, as equations can be equivalently rewritten in different forms. For instance, the equation $x^2 + 1 = 2$ can be rewritten as $x^2 - 1 = 0$, in which case $f(x) = x^2 - 1$ and $g(x) = 0$. This leads to the equivalence between solutions and roots. An $x^* \in \mathbb{R}$ is a *root* (or *zero*) of $f \colon \mathbb{R} \to \mathbb{R}$ if $f(x^*) = 0$. Graphically, roots are those points on the x-axis at which the function f intersects with the x-axis. Similarly, $\mathbf{x}^* \in \mathbb{R}^n$ is a root of $f \colon \mathbb{R}^m \to \mathbb{R}^n$ if $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$.

Theorem 5.2. *Let* $\mathbf{f}, \mathbf{g} \colon \mathbb{R}^n \to \mathbb{R}^m$. *An* $\mathbf{x}^* \in \mathbb{R}^n$ *is a solution to the equation* $\mathbf{f}(\mathbf{x}) = \mathbf{g}(\mathbf{x})$ *if and only if it is the root of the function* $\mathbf{f} - \mathbf{g}$.

Proof. Suppose that $\mathbf{x}^*$ solves the equation. Therefore $\mathbf{f}(\mathbf{x}^*) = \mathbf{g}(\mathbf{x}^*)$, which is equivalent to $\mathbf{f}(\mathbf{x}^*) - \mathbf{g}(\mathbf{x}^*) = \mathbf{0}$. This shows that $\mathbf{x}^*$ is a root of $\mathbf{f} - \mathbf{g}$. Conversely, if $\mathbf{x}^*$ is a root of $\mathbf{f} - \mathbf{g}$, it satisfies $\mathbf{f}(\mathbf{x}^*) - \mathbf{g}(\mathbf{x}^*) = \mathbf{0}$. This is equivalent to $\mathbf{f}(\mathbf{x}^*) = \mathbf{g}(\mathbf{x}^*)$, so that $\mathbf{x}^*$ solves the equation. □

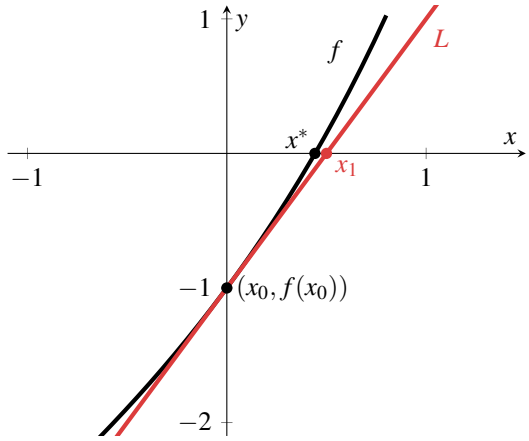


Figure 5.1: *The idea behind Newton's method*: The root x_1 of the linear approximation (i.e., tangent line) L_{x_0} for the function $f(x) = e^x + x - 2$ serves as an approximation for the root x^* of the function provided that the expansion point (i.e., point of tangency) x_0 is close to the actual root.

Therefore solving an equation is equivalent to finding roots of a function obtained by taking the difference of the sides of the original equation.

A function can have zero, one, or multiple roots. Equivalently, an equation can have zero, one, or multiple solutions. For example, the equation in Example 5.1 had two solutions ($x^* \pm 1$). The equation $x^2 = 0$ has one solution ($x^* = 0$) and the equation $x^2 + 1 = 0$ no solutions[3] (because x^2 is always non-negative). One can use Bolzano's theorem from basic analysis to ascertain if a continuous function has at least one solution.

Theorem 5.3 (Bolzano's theorem)**.** *Suppose that $f\colon [a,b] \to \mathbb{R}$ is continuous. If $f(a)$ and $f(b)$ have different signs, then $f(c) = 0$ for some $c \in [a,b]$.*

That $f(a)$ and $f(b)$ have different signs means that either (i) $f(a) < 0$ and $f(b) > 0$ or (ii) $f(a) > 0$ and $f(b) < 0$. These can be combined in the single condition $f(a)f(b) < 0$ because the product of a positive and a negative number is negative. Note that there may be roots on the interval $[a,b]$ even if the condition $f(a)f(b) < 0$ does not hold. Moreover, $f(a)f(b) < 0$ guarantees only that there is *at least one root* (i.e., there can be either one or multiple roots).

5.1 Newton's Method

Let us construct a root-finding method named after Newton. Recall from Section 3.2 that the linear approximation to a function f (or the Taylor polynomial of order one) at x_0 is the tangent line

$$L_{x_0}(x) = f(x_0) + f'(x_0)(x - x_0). \tag{5.1}$$

By Taylor's theorem (see Theorem 3.3 and Corollary 3.4), the linear approximation is a good approximation to f for x that are close to the expansion point x_0. Consequently, if f has a root near x_0, also the linear approximation has a root near x_0 and this root can be used to approximate the root of f. This is illustrated in Figure 5.1 for the function $f(x) = e^x + x - 2$ when $x_0 = 0$. If, in contrast to the previous case, the point of tangency x_0 is *not* close to the root of the function f, the root x_1 of the tangent line is still often closer to the root of the function than x_0. This means

[3] Unless one considers complex numbers, in which case the solutions are $x^* = \pm i$. We do not use complex numbers on this course.

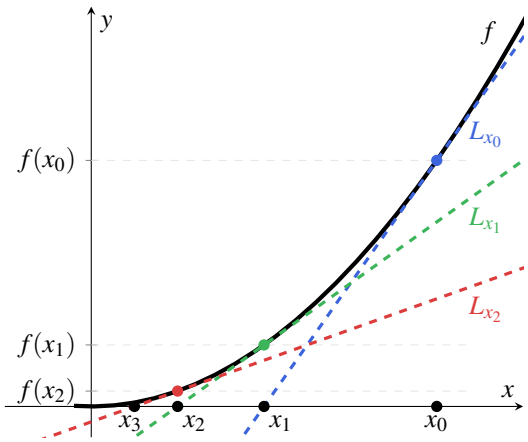


Figure 5.2: The roots x_k of the tangent lines of the function f approach the root of the function at $x = 0$.

that if tangent lines are iteratively formed at the root of the previous tangent line, these roots x_1, x_2, x_3, ... often approach the root of the function f. This is illustrated in Figure 5.2.

The roots of x_k of the tangent lines generally serve as increasingly accurate approximations for the root of the function. The resulting method is known as Newton's method. Let us derive an iteration formula that allows us to compute the next approximation, x_{k+1}, from the previous one, x_k. The root x_1 of the linear approximation in (5.1) formed at the point x_0 is given by

$$L_{x_0}(x_1) = 0 \quad \Longleftrightarrow \quad f(x_0) + f'(x_0)(x_1 - x_0) = 0 \quad \Longleftrightarrow \quad x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

By similar reasoning, the root x_2 of the tangent line formed at the point x_1 is given by

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)},$$

and, more generally, the root x_{k+1} of the tangent line formed at the point x_k is given by

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \tag{5.2}$$

Note that this formula assumes that $f'(x_k) \neq 0$. If $f'(x_k) = 0$, the linear approximation $L_{x_k}(x) = f(x_k) + f'(x_k)(x - x_k) = f(x_k)$ corresponds to a horizontal line that does not have a root, which means that one cannot proceed with the iteration as there is no next root x_{k+1} to construct a linear approximation at. With (5.2), new approximations can be calculated recursively once x_0 is selected. The choice of x_0 is often nothing but an informed guess. Accordingly, x_0 is called the initial guess. The closer the initial guess is to a root x^* of f, the more reliably the method works and the faster the approximations converge to the solution.

Definition 5.4 (Newton's method)**.** Let $f\colon \mathbb{R} \to \mathbb{R}$ be a differentiable function and $x_0 \in \mathbb{R}$ an *initial guess*. *Newton's method* is a sequence of approximations for a root of f given by the iteration

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \tag{5.3}$$

for $k \geq 0$.

Example 5.5. Let us use Newton's method with initial guess $x_0 = 4$ to compute approximations x_1 and x_2 to the root of the equation $x^2 = 0$, which is $x^* = 0$. Because $f(x) = x^2$ and $f'(x) = 2x$, the iteration formula (5.3) for Newton's method is

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_{k+1})} = x_k - \frac{x_k^2}{2x_k}. \tag{5.4}$$

Therefore

$$\begin{aligned} x_0 &= 4, \\ x_1 &= x_0 - \frac{f(x_0)}{f'(x_0)} = 4 - \frac{4^2}{2 \cdot 4} = 2, \\ x_2 &= x_1 - \frac{f(x_1)}{f'(x_1)} = 2 - \frac{2^2}{2 \cdot 2} = 1. \end{aligned}$$

In this case the iteration formula (5.4) simplifies to

$$x_{k+1} = x_k - \frac{x_k^2}{2x_k} = x_k - \frac{1}{2}x_k = \frac{1}{2}x_k,$$

so that

$$x_{k+1} = \frac{1}{2}x_k = \frac{1}{2} \cdot \frac{1}{2}x_{k-1} = \cdots = \frac{1}{2^{k+1}x_0}.$$

That is, in this case one could have computed each x_k directly from x_0 without iteration. *This happens very rarely in practice.*

The process described in the previous example, where the *same steps* are repeated over and over again, is called *iteration*. A single step of iteration is called an *iteration* or *iteration step*. The values calculated by iteration, such as x_k above, are called *iterates*. Therefore, Newton's method is based on iteration, and the formula in (5.3) is referred to as the Newton's method formula, the recursive formula, and also the *iteration formula*.

Newton's method does not work always. There are several potential issues:

- Although the iterates converge, they do not necessarily converge to a root.
- Geometrically it is evident that it can happen that the iterates keep repeating two consecutive values. This is the case, for example, with the function $f(x) = x^2 + x + 1$ when the initial guess or some subsequent iterate is 0 or -1.
- The iteration formula clearly fails if the derivative needs to be calculated at a point where it is zero.
- The iteration formula fails if the derivative needs to be calculated at a point where it does not exist, such as with the function $f(x) = |x^2 - 1| - 1$ at the points $x = \pm 1$. Less obviously, problems can also arise in the vicinity of such points, as with the function $f(x) = \sqrt[3]{x}$ around $x = 0$.
- In some cases, the iterates diverge if the initial guess is too far from the root, as in the case of the function $f(x) = x/(x^2 + 1)$.

The following theorem provides sufficient conditions to guarantee that Newton's method works.

Theorem 5.6. *Let $f(c) = 0$ and $f'(c) \neq 0$. Suppose that f is twice continuously differentiable on the interval $[c-a, c+a]$ for some $a > 0$. Then the iteration formula of Newton's method converges to c if the initial guess x_0 is sufficiently close to c.*

Proof. Omitted. □

Note that the theorem does not cover all functions for which Newton's method works. For example, for the function $f(x) = x^2$ we have $f'(x) = 0$, so that the assumptions of the theorem are not satisfies, but as seen above in Example 5.5, Newton's method nevertheless works for this function.

The theorem does not provide the exact value of c, but the information that the initial guess must be sufficiently close is still useful. For example, the function $f(x) = x/(x^2+1)$ satisfies the assumptions of the theorem, so that Newton's method will definitely work for it, as long as the initial guess is chosen appropriately. On the other hand, $f(x) = \sqrt[3]{x}$ does not satisfy the assumptions (why not?), so Newton's method may not work, even if the initial guess arbitrarily close to a root.

If Newton's method does not work on the first try, there is no reason to give up; instead, a different initial guess should be tried. The root can also be roughly determined first using another root-finding method. For this reason, more advanced algorithms, such as MATLAB's `fzero` function, apply multiple methods.

5.2 Newton's Method for Systems of Equations

Newton's method for systems of equations is almost identical to the single-variable case when formulated using vector-valued functions. For this purpose, one must be familiar with the concept of the *Jacobian matrix*, which is a generalization of the gradient for vector-valued functions. Before introducing the Jacobian matrix, we will also review other necessary concepts.

5.2.1 Nonlinear Systems of Equations

A system of equations is linear if it has the form

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \tag{5.5}$$
$$a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \tag{5.6}$$
$$\vdots \tag{5.7}$$
$$a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m, \tag{5.8}$$

where a_{ij} and b_i are constant and x_j variables. In other cases, a system of equations is *nonlinear*. A system of equations is considered nonlinear if it contains at least one term that is neither a constant nor of the form constant · variable. For example, the system of equations

$$\begin{aligned} 4xy + y &= 1 \\ x - 2y &= 3 \end{aligned}$$

is nonlinear due to the term $4xy$ and the system of equations

$$\begin{aligned} 4x + y &= 1 \\ x - 2\sin(y) &= 3 \end{aligned}$$

due to the term $-2\sin(y)$.

Linear systems of equations can be written in the form $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is a constant matrix, and they always have exactly 1, 0, or infinitely many solutions. They can all be solved using

Gaussian elimination, and there also exist numerical solution methods that are more efficient than Gaussian elimination and Newton's method.

Nonlinear systems of equations cannot be written in the form $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is a constant matrix, and the number of their solutions can be other than 1, 0, or infinite. Solving them is significantly more difficult than solving linear systems of equations.

5.2.2 Roots of a Multivariate Function

The roots of a single-variable function are the points on the x-axis where the function's graph, which is often a curve, intersects the x-axis. These are typically distinct points and are often finite in number.

The roots of a real-valued function of two variables are the points in the xy-plane where the function's graph, which is often a surface, intersects the xy-plane. These usually form a curve, meaning that there are infinitely many such points. For example the roots of the function $f(x,y) = x^2 + y^2 - 1$ satisfy

$$\begin{aligned} f(x,y) &= 0 \\ x^2 + y^2 - 1 &= 0 \\ x^2 + y^2 &= 1, \end{aligned}$$

so they form a circle of radius 1 centered at the origin. In the case of two variables, instead of referring to roots, one may also speak of level curves given by $f(x,y) = 0$, which represent the curves in the xy-plane where the function's height is zero (compare level curves to contour lines on maps).

The roots of a real-valued function of three variables generally form surfaces. For example, the zeroes of the function $f(x,y,z) = x^2 + y^2 + z^2 - 1$ form a sphere of radius 1 centered at the origin, which is given by $x^2 + y^2 + z^2 = 1$. In the case of three variables one may also refer to level surfaces given by $f(x,y,z) = 0$ instead of roots.

5.2.3 Tangent Plane

For functions of two variables, the concept corresponding to the tangent line is the tangent plane. The linear approximation, or tangent plane, or first-degree Taylor polynomial, of a function f at a point (x_0, y_0) is given by

$$L(x,y) = f(x_0,y_0) + f_x(x_0,y_0)(x - x_0) + f_y(x_0,y_0)(y - y_0).$$

The roots of a function of two variables can be approximated by the zeroes of the tangent plane formed near the actual zero, similarly to how the zeroes of a single-variable function are approximated by the zeroes of the tangent line. However, the difference is that only a portion of the zeroes of the tangent plane, which form a line, provide a good approximation of some of the roots. The accompanying figure shows the function $f(x,y) = 3 - x^2 - y^2$ along with the tangent plane $L(x,y) = 1 - 2(x-1) - 2(y-1)$ at the point $(1,1)$ and its roots. Note that only some of the points on the line provide a good approximation of some of the function's roots.

5.2.4 Vector-Valued Functions

Vector-valued functions are functions of one or more variables whose values are vectors. For example,

$$\mathbf{f}\colon \mathbb{R} \to \mathbb{R}^3, \quad \mathbf{f}(t) = \begin{bmatrix} t^2 \\ t^2 \\ 3 \end{bmatrix}$$

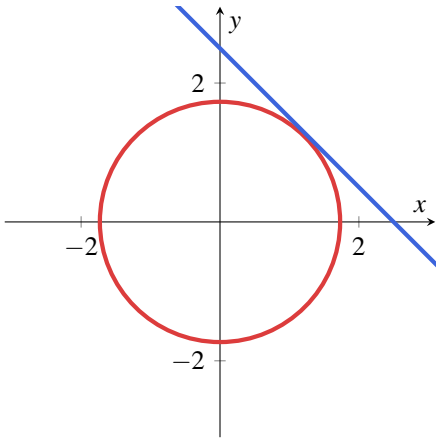


Figure 5.3: The function $f(x,y) = 3 - x^2 - y^2$ along with the tangent plane $L(x,y) = 1 - 2(x-1) - 2(y-1)$ at the point $(1,1)$ and its roots.

is a vector-valued function of one variable whose values are three-dimensional vectors. Such a function could, for instance, describe the (three-dimensional) position of an object as a function of time t. On the other hand, the function

$$\mathbf{f}\colon \mathbb{R}^3 \to \mathbb{R}^3, \quad \mathbf{f}(x,y,z) = \begin{bmatrix} xz \\ y+1 \\ x-2z+\sin(y) \end{bmatrix}$$

is a vector-valued function of three variables whose values are three-dimensional vectors. Such a function could, for example, represent the wind velocity vector as a function of position (x,y,z).

In general, vector-valued functions have the form

$$\mathbf{f}\colon D \to \mathbb{R}^n,\ D \subset \mathbb{R}^m, \quad \mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_n(\mathbf{x}) \end{bmatrix},$$

and the functions f_i are called *component functions*. We can also write $\mathbf{f}(x_1, x_2, \ldots, x_m)$, but using vector notation for variables is clearly more concise in the case of multiple variables.

The roots of a vector-valued function refer to the set of input values for which $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ holds. From this it follows that solving a system of equations can equivalently be interpreted as solving for the roots of a vector-valued function. For example, we can write

$$\begin{aligned} x^2 + y^2 &= 3 \\ (x-2)^2 + (y-1)^2 &= 2, \end{aligned}$$

which is equivalent to

$$\begin{aligned} x^2 + y^2 - 3 &= 0 \\ (x-2)^2 + (y-1)^2 - 2 &= 0 \end{aligned}$$

which is equivalent to

$$f(x,y) = 0$$

$$g(x,y) = 0,$$

and define the function

$$\mathbf{f}(x,y) = \begin{bmatrix} f(x,y) \\ g(x,y) \end{bmatrix} = \begin{bmatrix} x^2 + y^2 - 3 \\ (x-2)^2 + (y-1)^2 - 2 \end{bmatrix},$$

so that the original pair of equations can be viewed as the equation $\mathbf{f}(x,y) = \mathbf{0}$.

When generalizing Newton's method for systems of equations, we think of solving for the roots of a vector-valued function. The iteration formula becomes almost identical to the single-variable case. In this case, x_k and x_{k+1} are replaced by vectors $\mathbf{x}_k$ and $\mathbf{x}_{k+1}$, and the real-valued function f is replaced by the vector-valued function $\mathbf{f}$.

5.2.5 Jacobian Matrix

The *gradient* of a real-valued function $f\colon D \to \mathbb{R}$, $D \subset \mathbb{R}^2$ of two variables is the function

$$\nabla f(x,y) = \begin{bmatrix} f_x(x,y) \\ f_y(x,y) \end{bmatrix},$$

which generalizes the derivative. Generally, the gradient of a function $f\colon D \to \mathbb{R}$, $D \subset \mathbb{R}^n$ of multiple variables is the function

$$\nabla f(\mathbf{x}) = \begin{bmatrix} f_{x_1}(\mathbf{x}) \\ f_{x_2}(\mathbf{x}) \\ \vdots \\ f_{x_n}(\mathbf{x}) \end{bmatrix}.$$

The *Jacobian matrix* of a vector-valued function

$$\mathbf{f}\colon D \to \mathbb{R}^2,\ D \subset \mathbb{R}^2, \quad \mathbf{f}(x,y) = \begin{bmatrix} f(x,y) \\ g(x,y) \end{bmatrix},$$

of two variables is the matrix-valued function

$$\mathrm{J}_{\mathbf{f}}(x,y) = \begin{bmatrix} f_x(x,y) & f_y(x,y) \\ g_x(x,y) & g_y(x,y) \end{bmatrix}.$$

For example, the function

$$\mathbf{f}(x,y) = \begin{bmatrix} x^2 y \\ x + 2y^3 \end{bmatrix}$$

has the Jacobian matrix

$$\mathrm{J}_{\mathbf{f}}(x,y) = \begin{bmatrix} 2xy & x^2 \\ 1 & 6y^2 \end{bmatrix}.$$

Generally, the Jacobian matrix of a vector-valued function

$$\mathbf{f}\colon D \to \mathbb{R}^m,\ D \subset \mathbb{R}^n, \quad \mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix},$$

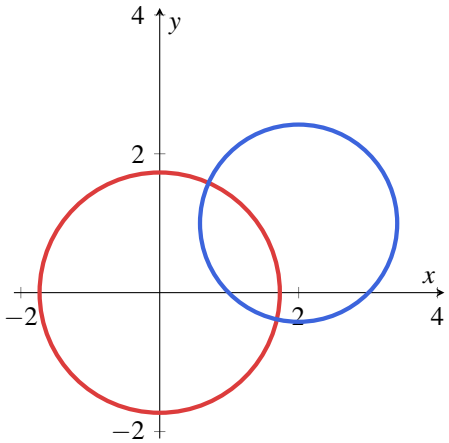


Figure 5.4: Equations $x^2+y^2=3$ and $(x-2)^2+(y-1)^2=2$ hold simultaneously only when the corresponding curves intersect.

of multiple variables is the matrix-valued function

$$\mathrm{J}_{\mathbf{f}}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}.$$

When Newton's method is later extended to systems of equations, the division by the derivative $f'(x_k)$ in the single-variable iteration formula $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$ is replaced by multiplication by the inverse of the Jacobian matrix $\mathrm{J}_{\mathbf{f}}(\mathbf{x})^{-1}$.

5.2.6 Geometric Idea of Newton's Method for a Pair of Equations

Solving a system of two equations involves finding the intersection points of level curves. For example, for the pair of equations

$$x^2+y^2=3 \tag{5.9}$$

$$(x-2)^2+(y-1)^2=2 \tag{5.10}$$

the solutions are the intersection points of the curves in the accompanying figure. To understand why Newton's method can also be applied to solving pairs of equations, we need to look at the pair from a slightly different perspective.

The previous pair of equations can be written as

$$x^2+y^2-3=0$$
$$(x-2)^2+(y-1)^2-2=0,$$

where the left-hand sides of the equations can be interpreted as the functions $f(x,y)=x^2+y^2-3$ and $g(x,y)=(x-2)^2+(y-1)^2-2$. Interpreted in this way, the pair

$$f(x,y)=0$$

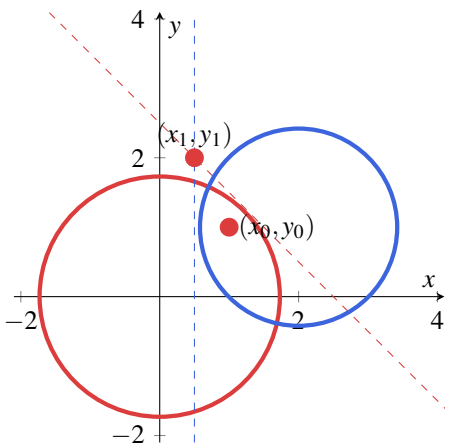


Figure 5.5: The intersection point of the lines formed by the roots of the tangent planes of the functions $f(x,y) = x^2 + y^2 - 3$ and $g(x,y) = (x-2)^2 + (y-1)^2 - 2$ can be used to approximate a common root of the functions f and g.

$$g(x,y) = 0$$

seeks the common roots of the bivariate functions f and g.

The roots of a function of two variables can be approximated by the roots of the tangent plane formed at a point (x_0, y_0) near the root, similarly to how the roots of a single-variable function are approximated by the roots of the tangent line. Thus, the common roots of functions f and g can be approximated by the intersection points of the roots of the tangent planes formed for the functions. The accompanying figure shows the roots of the functions f and g as well as the roots of the tangent planes formed at the point $(x_0, y_0) = (1,1)$. The intersection point of the lines, (x_1, y_1), roughly approximates the upper root of the system of equations (5.9).

When new tangent planes are formed at the intersection point of the lines (x_1, y_1) from the previous figure, and the intersection point (x_2, y_2) of the new lines formed by the roots of these tangent planes is calculated, a better approximation for the solution of the system of equations is obtained. This process can be repeated indefinitely, and often the intersection points (x_k, y_k) converge towards one of the solutions of the system. This is the geometric idea of Newton's method for a system of two equations. Next, let us move on to examine the problem computationally.

5.2.7 The Formula of Newton's Method for Systems of Equations

Let us derive the formula for Newton's method in the case of a system of two equations. All systems of two equations can be written in the form

$$\begin{aligned} f(x,y) &= 0 \\ g(x,y) &= 0. \end{aligned}$$

We approximate the solution of the system of equations using tangent planes in the way described in the previous section. We begin by defining the tangent planes at the point (x_0, y_0). The equations of the tangent planes are

$$L_f(x,y) = f(x_0,y_0) + f_x(x_0,y_0)(x - x_0) + f_y(x_0,y_0)(y - y_0)$$

and

$$L_g(x,y) = g(x_0,y_0) + g_x(x_0,y_0)(x - x_0) + g_y(x_0,y_0)(y - y_0).$$

The intersection lines of these planes with the xy-plane are given by $L_f(x,y) = 0$ and $L_g(x,y) = 0$. The solution of the system of equations can be approximated by the intersection point of these lines. This can be solved from the system of equations

$$\begin{aligned} L_f(x,y) &= 0 \\ L_g(x,y) &= 0 \end{aligned}$$

which is equivalent to

$$\begin{aligned} f(x_0,y_0) + f_x(x_0,y_0)(x-x_0) + f_y(x_0,y_0)(y-y_0) &= 0 \\ g(x_0,y_0) + g_x(x_0,y_0)(x-x_0) + g_y(x_0,y_0)(y-y_0) &= 0, \end{aligned}$$

which becomes

$$\begin{aligned} f_x(x_0,y_0)x + f_y(x_0,y_0)y &= f_x(x_0,y_0)x_0 + f_y(x_0,y_0)y_0 - f(x_0,y_0) \\ g_x(x_0,y_0)x + g_y(x_0,y_0)y &= g_x(x_0,y_0)x_0 + g_y(x_0,y_0)y_0 - g(x_0,y_0), \end{aligned}$$

when the variables and constants are moved to their respective sides. In these equations, x and y are the unknowns and all other coefficients and terms are constants.

The system of equations could be solved directly, but before doing so, let us note that the system appears to be highly regular in a certain way way. This regularity becomes clearer when we write the system using matrices:

$$\begin{bmatrix} f_x(x_0,y_0) & f_y(x_0,y_0) \\ g_x(x_0,y_0) & g_y(x_0,y_0) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} f_x(x_0,y_0) & f_y(x_0,y_0) \\ g_x(x_0,y_0) & g_y(x_0,y_0) \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} - \begin{bmatrix} f(x_0,y_0) \\ g(x_0,y_0) \end{bmatrix}.$$

This can be written even more conveniently if we adopt the notation

$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}, \quad \mathbf{x}_0 = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} \quad \text{and} \quad \mathbf{f}(\mathbf{x}) = \begin{bmatrix} f(x,y) \\ g(x,y) \end{bmatrix}.$$

We also observe that the coefficient matrix is actually the Jacobian matrix of the function $\mathbf{f}$, meaning that we have

$$\begin{bmatrix} f_x(x_0,y_0) & f_y(x_0,y_0) \\ g_x(x_0,y_0) & g_y(x_0,y_0) \end{bmatrix} = \mathbf{J_f}(\mathbf{x}).$$

With this notation the matrix equation becomes

$$\mathbf{J_f}(\mathbf{x}_0)\mathbf{x} = \mathbf{J_f}(\mathbf{x}_0)\mathbf{x}_0 - \mathbf{f}(\mathbf{x}_0),$$

and if we assume that the Jacobian matrix is invertible, the solution of the system of equations is

$$\mathbf{x} = \mathbf{x}_0 - \mathbf{J_f}^{-1}(\mathbf{x}_0)\mathbf{f}(\mathbf{x}_0).$$

The iterate proceeding the initial guess $\mathbf{x}_0$ is thus $\mathbf{x}_1 = \mathbf{x}_0 - \mathbf{J_f}^{-1}(\mathbf{x}_0)\mathbf{f}(\mathbf{x}_0)$ and the next one $\mathbf{x}_2 = \mathbf{x}_1 - \mathbf{J_f}^{-1}(\mathbf{x}_1)\mathbf{f}(\mathbf{x}_1)$, and so on. In general we thus have $\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{J_f}^{-1}(\mathbf{x}_k)\mathbf{f}(\mathbf{x}_k)$.

The reasoning above generalizes in a similar way to $n \times n$ systems of equations.

Definition 5.7 (Newton's method for systems of equations)**.** In Newton's method, approximate solutions to the system of equations $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ are calculated using the iteration

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{J_f}^{-1}(\mathbf{x}_k)\mathbf{f}(\mathbf{x}_k),$$

starting from a user-specified vector $\mathbf{x}_0$, which is called the initial guess.

Note that the formula for Newton's method for systems of equations is almost identical to the case of a single variable, the only difference being that the functions and variables are now vectors, and the derivative is replaced by the Jacobian matrix. Specifically, it is important to remember that the multiplication by the Jacobian matrix must happen *from the left*.

Example 5.8. Let us use Newton's method to find an approximation $\mathbf{x}_1$ to the solution of the system of equations

$$\begin{aligned} x^2 + y^2 &= 3 \\ (x-2)^2 + (y-1)^2 &= 2 \end{aligned}$$

when the initial guess is $\mathbf{x}_0 = (1,1)$. Information necessary for iteration:

$$\mathbf{f}(x,y) = \begin{bmatrix} f(x,y) \\ g(x,y) \end{bmatrix} = \begin{bmatrix} x^2 + y^2 - 3 \\ (x-2)^2 + (y-1)^2 - 2 \end{bmatrix}.$$

$$\mathbf{J_f}(x,y) = \begin{bmatrix} 2x & 2y \\ 2(x-2) & 2(y-1) \end{bmatrix}$$

$$\mathbf{J_f}^{-1}(x,y) = \frac{1}{4x(y-1) - 4y(x-2)} \begin{bmatrix} 2(y-1) & -2y \\ -2(x-2) & 2x \end{bmatrix}.$$

First iterate:

$$\begin{aligned} \mathbf{x}_1 &= \mathbf{x}_0 - \mathbf{J_f}^{-1}(\mathbf{x}_0)\mathbf{f}(\mathbf{x}_0) \\ &= \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \frac{1}{4(1-1) - 4(1-2)} \begin{bmatrix} 2(1-1) & -2 \\ -2(1-2) & 2 \end{bmatrix} \begin{bmatrix} 1^2 + 1^2 - 3 \\ (1-2)^2 + (1-1)^2 - 2 \end{bmatrix} \\ &= \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \frac{1}{4} \begin{bmatrix} 0 & -2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} \\ &= \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \frac{1}{4} \begin{bmatrix} 2 \\ -4 \end{bmatrix} \\ &= \begin{bmatrix} \frac{1}{2} \\ 2 \end{bmatrix}. \end{aligned}$$

5.3 Fixed Point Method

Fixed point method is numerical equation-solving method similar to Newton's method.

5.3.1 Fixed Point

Usually a function maps an input to an output that does not equal the input. For example, the function $f(x) = x^2$ maps $x = 2$ to the $f(x) = 4$. This function also has two variable values that remain unchanged under the mapping, namely $x = 0$ and $x = 1$. These are the fixed points of the function f.

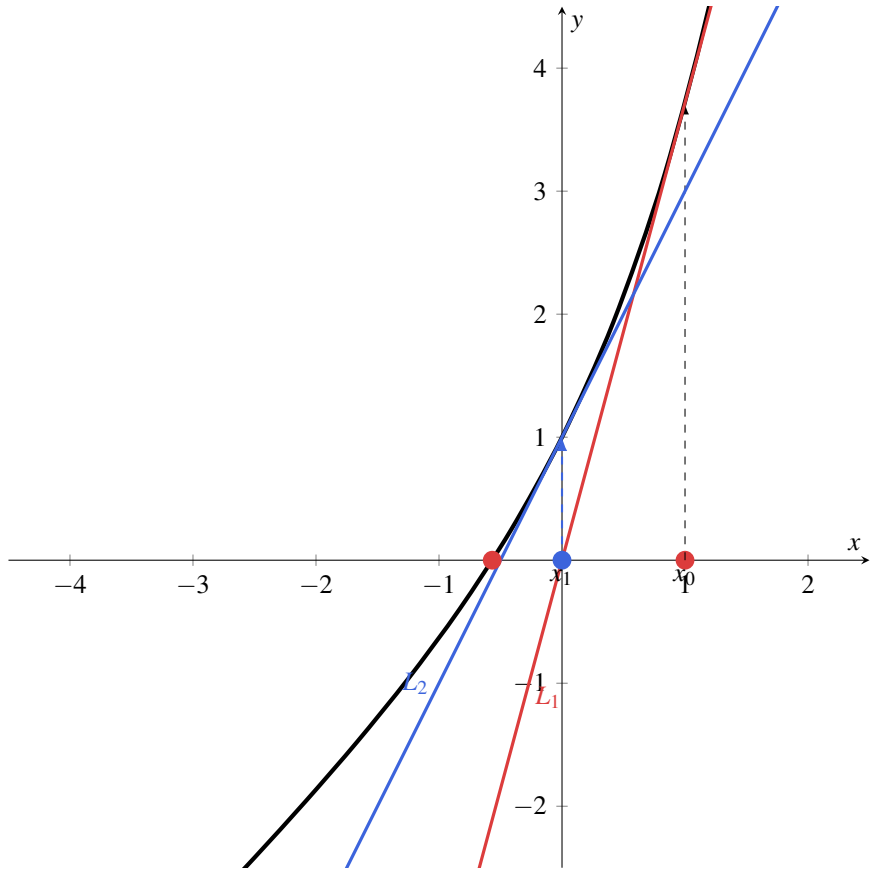
Definition 5.9 (Fixed point)**.** The fixed-points of a function $f\colon D \to \mathbb{R}$, where $D \subset \mathbb{R}$, are points $c \in D$ for which $f(c) = c$.

A function can have multiple fixed points, as in the previous example, or it may have none at all. The geometric interpretation of a fixed point is the x-coordinate of the intersection point between the line $y = x$ and the graph of the function f (which is, in turn, the same as the y-coordinate of that point). The fixed points of a function can be determined by solving the equation $f(x) = x$.

5.3.2 Geometric Intuition

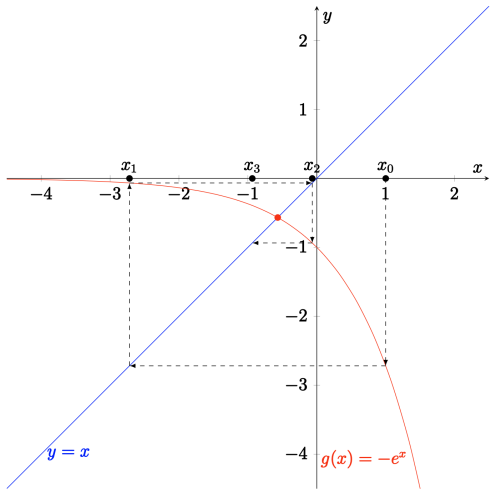
Just like Newton's method, the fixed point method is based on a highly intuitive geometric idea. To begin, let us note for comparison that the equation $e^x = -x$ is solved graphically using Newton's method with the following steps, which are illustrated in the accompanying figure:

1. The equation is rewritten in the form $e^x + x = 0$, meaning that all terms are moved to the same side.
2. The graph of the function $f(x) = e^x + x$ is drawn, so that the solutions of the original equation correspond to the roots of the function f.
3. An initial guess x_0 is chosen as the first approximation to the root of the function f.
4. A tangent is drawn for the function f at the point x_0.
5. The zero of the tangent line, denoted as x_1, is determined. This becomes the second approximation to the root of the function f.
6. The process of drawing tangent lines is repeated as described in steps 4 and 5, always replacing the previous iterate x_k with the next one, x_{k+1}.
7. In certain cases, the iterates converge towards the root of the function f.



The following stesps, illustrated in the accompanying figure, are used to solve the equation $e^x = -x$ graphically with the fixed point method:

1. The equation is rewritten in the form $x = -e^x$, so that one side contains only x. There are different ways to do this. We will discuss these later.
2. A line $y = x$ and the graph of the function $g(x) = -e^x$ (i.e., graphs of both sides of the equation from step 2) are drawn. Then the solutions of the original equation are the x-coordinates of the intersection points of these curves.
3. An initial guess x_0 is chosen as the first approximation to the solution.
4. At point x_0, move vertically to the height $g(x_0)$.
5. Move horizontally at height $g(x_0)$ to the line $y = x$, that is, to the point $(g(x_0), g(x_0))$. The x-coordinate of the point becomes a new approximation x_1 to the solution to the equation.
6. Repeat the vertical and horizontal shifts of steps 4 and 5, replacing the previous iterate x_k with the new one, x_{k+1}.
7. In certain cases, the iterates converge to the solution of the equation.



When using the fixed point method above, the equation $e^x = -x$ could also have been solved in the form $x = \ln(-x)$, where the considered function would have been $g(x) = \ln(-x)$. This is an example of how the function g is not unique, as it can actually be chosen in infinitely many different ways. For example, the above can be written as

$$\begin{aligned} -x &= e^x \\ ax - x &= e^x + ax \\ x &= \frac{e^x + ax}{a-1} \end{aligned}$$

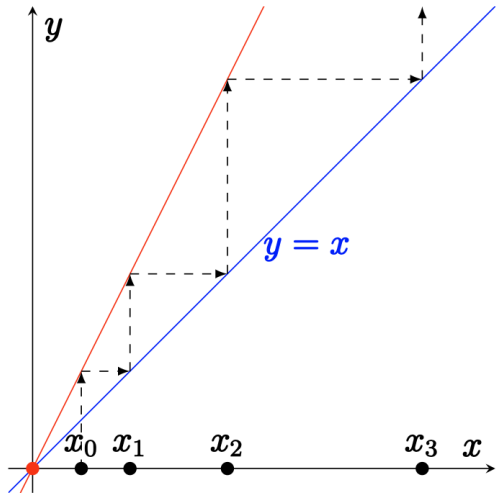
so that each value of the parameter a corresponds to the function

$$g(x) = \frac{e^x + ax}{a-1}.$$

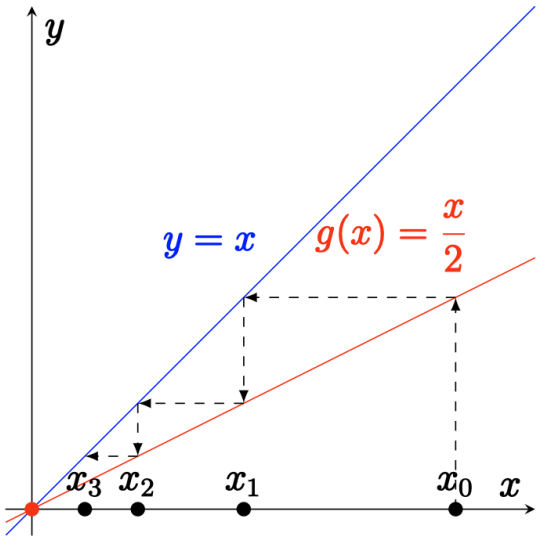
It was noted above that the iterates converge to the solution *in certain situations*. The fixed point method does not always work, just as the Newton method does not always work. Solving the equation $-x = e^x$ with the choice $g(x) = \ln(-x)$ fails because it requires examining the function g at points where it is not defined (Draw a graph and try different initial guesses). However, iteration can also fail even if g is defined everywhere. For example, if the equation

$$x - \frac{x}{2} = 0$$

is solved by first writing it as $x = 2x$, so that $g(x) = 2x$, the iterates diverge as shown in the accompanying figure.



If we instead try to solve the same equation by first writing it as $x = \frac{x}{2}$, where $g(x) = \frac{x}{2}$, then the iterates converge as shown in the accompanying figure.



An obvious difference between the functions g that behaved differently above is at least that the first one is steeper than the second. By repeating similar analyses, it can be observed that the steepness of the function g appears to be a critical factor for convergence. The fixed point method can also fail because the iterates do not converge to the correct value. However, this can happen only when the function g is not continuous.

Theorem 5.10. *Let g be a continuous function. If the iterates of the fixed point iteration $x_{k+1} = g(x_k)$ converge, they converge to a fixed point.*

Proof. Omitted. □

The following function serves as a counter-example for the necessity of continuity of g:

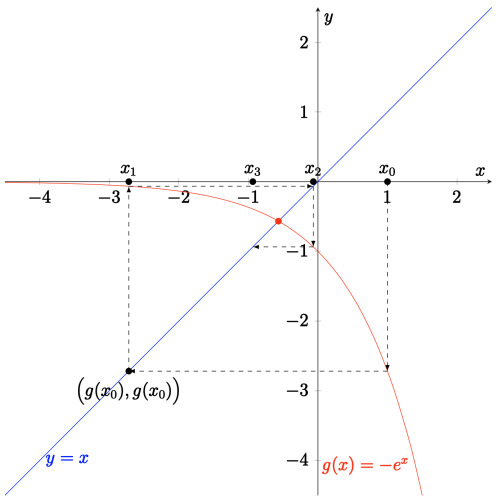
$$g(x) = \begin{cases} \frac{x}{2}, & x \neq 0 \\ -1, & x = 0 \end{cases}.$$

5.3.3 *Iteration Formula*

Let us derive the iteration formula for the fixed point method. The goal is to deduce how x_{k+1} can be computed from on the previous value x_k (or from on multiple previous values such as x_k and x_{k-1} if this proves necessary). For comparison, recall that in Newton's method, the iteration formula was of the form

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

We begin by examining the first few iterations as shown in the figure shown earlier.



The initial guess x_0 is chosen arbitrarily. Then, we move to the height $g(x_0)$, move to the line $y = x$ at the point $(g(x_0), g(x_0))$, and take the horizontal coordinate of this point as the new iterate x_1. Thus, we have $x_1 = g(x_0)$. By similar reasoning, we observe that $x_2 = g(x_1)$, $x_3 = g(x_2)$, and so on. That is, in general $x_{k+1} = g(x_k)$.

Definition 5.11 (Fixed point method)**.** In the *fixed point method*, a root of the function $x - g(x)$ is approximated with the iteration formula

$$x_{k+1} = g(x_k),$$

which is initialized with the initial guess x_0.

Let us take a simple but illustrative example that has already been examined graphically above.

Example 5.12. Let us solve the equation

$$x - \frac{x}{2} = 0$$

with the fixed point method. An obvious choice is to rewrite the equation in the form $x = \frac{x}{2}$,

which gives the iteration formula $x_{k+1} = \frac{x_k}{2}$, so that $g(x) = \frac{x}{2}$. If the initial guess is chosen as $x_0 = 4$, we get

$$\begin{aligned}
x_0 &= 4 \\
x_1 &= g(x_0) = g(4) = \frac{4}{2} = 2 \\
x_2 &= g(x_1) = g(2) = \frac{2}{2} = 1 \\
x_3 &= g(x_2) = g(1) = \frac{1}{2} \\
&\vdots
\end{aligned}$$

We obtain a sequence that seems to converge to zero. By substituting or solving, we can check that this is the correct solution. Also, refer to the figure from the previous chapter or draw your own figure.

Another obvious choice is to rewrite the equation in the form $x = 2x$, which gives the iteration formula $x_{k+1} = 2x_k$. In this case, the iteration diverges for any initial guess that is not zero. For example, in the case $x_0 = 4$, we get

$$\begin{aligned}
x_0 &= 4 \\
x_1 &= g(x_0) = g(4) = 2 \cdot 4 = 8 \\
x_2 &= g(x_1) = g(8) = 2 \cdot 8 = 16 \\
x_3 &= g(x_2) = g(16) = 2 \cdot 16 = 32 \\
&\vdots
\end{aligned}$$

A quick examination of the fixed-point method can be done, similar to Newton's method, by storing the initial guess in the variable `ans` and repeatedly executing the command `g(ans)`. A more careful examination is best done using a loop.

Newton's method can actually be interpreted as a fixed point method, where

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

This allows for the treatment of the theory of Newton's method using the theory of the fixed point method, which is not addressed in this course.

5.3.4 On Convergence

It was noted above that the fixed point iteration seems to converge when the graph of the function g rises or falls sufficiently gently. The following theorem provides a precise condition.

Theorem 5.13. *Let c be a fixed point of a function g. Suppose that g is differentiable at the point c. If $\left|g'(c)\right| < 1$, then the fixed point iteration converges, provided that x_0 is chosen sufficiently close to the fixed point. In this case, the fixed point is called attractive or attracting. On the other hand, if $\left|g'(c)\right| > 1$, then the fixed point iteration converges only in the case where $x_n = c$. In this case, the fixed point is called repulsive or repelling.*

Proof. Omitted. □

We can illustrate the connection between the change in iterates and the derivative. For the iteration to converge, the horizontal shifts seen in the figures should decrease. This means that

we should have

$$\frac{|x_{k+2} - x_{k+1}|}{|x_{k+1} - x_k|} < K$$

for some $K \in (0,1)$, so that the next horizontal move is smaller than the previous one. We can rewrite this condition as

$$\left| \frac{g(x_{k+1}) - g(x_k)}{x_{k+1} - x_k} \right| < K,$$

which means that the average rate of change of the interval (x_k, x_{k+1}) (or (x_{k+1}, x_k) if $x_k > x_{k+1}$) must be less than one in absolute value. If $|g'(c)| < 1$, so that the instantaneous rate of change at c is less than one, then it is possible to find an interval containing c such that the average rate of change on that interval is less than one.

5.4 Jacobi Method for Linear Systems

The solutions to systems of *linear equations* fall into three categories: a unique solution, multiple solutions, or no solution. Regardless of which of these three types the solution belongs to, it can always be determined straightforwardly using Gaussian elimination. Because of this, one might think that solving systems of linear equations does not require any special numerical theory as one can simply let a computer determine the solution using Gaussian elimination. On the other hand, solving systems of linear equations is computationally expensive, with the computational effort growing *cubically with the size of the system*. This is what creates the need for numerical solution methods. For sufficiently large matrices, computational budget inevitably becomes insufficient. Another reason is that rounding errors can significantly affect the final result. This necessitates the development of more efficient and accurate methods for solving systems of linear equations.

A numerical method finds only one solution at a time for an equation or a system of equations. Therefore, it cannot determine the multiple solutions of a linear system that includes free parameters. Instead, the method merely outputs a collection of decimal numbers approximating the solution. For this reason we focus only on systems that have a unique solution. In practice, this means $n \times n$ systems, since in the case of a low-rank matrix, the solution can never be unique (too few equations), and in the case of a high-rank matrix, a unique solution is quite rare (too many equations). Thus, unless otherwise stated, the matrix that determines the linear system can be assumed to be a square matrix in this chapter.

5.4.1 Direct and Iterative Methods

Newton's method does not immediately yield a good approximation to the solution of an equation. Instead, a sequence of approximations is generated, which (under certain conditions) converge to the exact solution. In contrast, when solving a system of linear equations numerically using Gaussian elimination, only a single approximation to the solution is obtained (which may be exact in exceptional cases). Gaussian elimination is not an iterative process but an example of a *direct method*, whereas Newton's method is an example of an *iterative method*. A direct method is thus a non-iterative method that, in theory, provides an exact solution, but in practice outputs in an approximation due to factors such as rounding errors and the limited precision of real number representation in computers.

There are both direct and iterative methods for solving systems of linear equations. This course focuses on iterative methods. Direct methods are also covered to provide a better overall understanding of the topic, but a more detailed treatment of a significant direct method, the LU decomposition, is reserved for other courses. Systems of linear equations can be solved not only using the Gaussian elimination algorithm and its variations but also by using the Gaussian matrix and Cramer's rule. All of these are direct methods, and although they theoretically always

yield an exact solution, in practice, when computed with floating-point arithmetic, the solution is merely an approximation.

5.4.2 The Jacobi Method

Let us try to solve the system of equations

$$\begin{aligned} 4x_1 + 2x_2 + x_3 &= 8 \\ x_1 + 5x_2 + 3x_3 &= 12 \\ x_1 + x_2 + 3x_3 &= 8 \end{aligned}$$

using the fixed point method. The goal is to rewrite the system of equations in the form $\mathbf{x} = \mathbf{g}(\mathbf{x})$. There are infinitely many ways to do this, but for reasons that will become clear later, we choose to solve for the variable corresponding to the diagonal element in each row:

$$\begin{aligned} 4x_1 + 2x_2 + x_3 &= 8 \\ x_1 + 5x_2 + 3x_3 &= 12 \\ x_1 + x_2 + 3x_3 &= 8 \end{aligned}$$

which is equivalent with

$$\begin{aligned} 4x_1 &= 8 - 2x_2 - x_3 \\ 5x_2 &= 12 - x_1 - 3x_3 \\ 3x_3 &= 8 - x_1 + x_2 \end{aligned}$$

which is equivalent with

$$\begin{aligned} x_1 &= \frac{8}{4} - \frac{2}{4}x_2 - \frac{1}{4}x_3 \\ x_2 &= \frac{12}{5} - \frac{1}{5}x_1 - \frac{3}{5}x_3 \\ x_3 &= \frac{8}{3} - \frac{1}{3}x_1 - \frac{1}{3}x_2. \end{aligned}$$

This choice gives the following iteration formula for the fixed point method:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_{k+1} = \begin{bmatrix} 2 \\ \frac{12}{5} \\ \frac{8}{3} \end{bmatrix} + \begin{bmatrix} 0 & \frac{1}{2} & -\frac{1}{4} \\ \frac{1}{5} & 0 & -\frac{3}{5} \\ \frac{1}{3} & -\frac{1}{3} & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_k.$$

When the formula is tested, it is observed that the iterates converge to the correct solution $\mathbf{x} = \begin{bmatrix} 1 & 1 & 2 \end{bmatrix}^\mathsf{T}$ regardless of the initial guess. For example, in the case $\mathbf{x}_0 = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^\mathsf{T}$ we have

$$\mathbf{x}_1 \approx \begin{bmatrix} 2.0000 \\ 2.4000 \\ 2.6667 \end{bmatrix}, \quad \mathbf{x}_2 \approx \begin{bmatrix} 0.1333 \\ 0.4000 \\ 1.2000 \end{bmatrix}, \quad \ldots, \quad \mathbf{x}_{35} \approx \begin{bmatrix} 1.0000 \\ 1.0000 \\ 2.0000 \end{bmatrix}.$$

A Convergence Criterion When the system of equations discussed earlier is written in the form $\mathbf{x} = \mathbf{g}(\mathbf{x})$ in a different way than previously, the fixed point method may no longer converge. For example, if we use the simplest choice

$$4x_1 + 2x_2 + x_3 = 8$$

$$\begin{aligned} x_1 + 5x_2 + 3x_3 &= 12 \\ x_1 + x_2 + 3x_3 &= 8 \end{aligned}$$

which is equivalent with

$$\begin{aligned} x_3 &= 8 - 4x_1 - 2x_2 \\ x_1 &= 12 - 5x_2 - 3x_3 \\ x_2 &= 8 - x_1 - 3x_3 \end{aligned}$$

which is equivalent with

$$\begin{aligned} x_1 &= 12 - 5x_2 - 3x_3 \\ x_2 &= 8 - x_1 - 3x_3 \\ x_3 &= 8 - 4x_1 - 2x_2 \end{aligned}$$

which gives the iteration formula

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_{k+1} = \begin{bmatrix} 12 \\ 8 \\ 8 \end{bmatrix} + \begin{bmatrix} 0 & -5 & -3 \\ -1 & 0 & -3 \\ -4 & -2 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_k ,$$

then the iterates diverge for the initial guess $\mathbf{x}_0 = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^\mathsf{T}$:

$$\mathbf{x}_1 = \begin{bmatrix} 12 \\ 8 \\ 8 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} -52 \\ -28 \\ -56 \end{bmatrix}, \quad \ldots, \quad \mathbf{x}_3 = \begin{bmatrix} 320 \\ 228 \\ 272 \end{bmatrix}.$$

This happens with this iteration formula for any initial guess. On the other hand, solving for the variables corresponding to the diagonal elements, which was found to work with the previous system of equations, does not always work. For example, for the linear system of equations

$$\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

we obtain the iteration formula

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}_{k+1} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 3 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}_k$$

which diverges for every initial guess $\mathbf{x}_0$.

These observations raise the question about the cases in which the iteration converges. If for each row the absolute value of the diagonal element is greater than the sum of the absolute values of the other coefficients in that row, then the iteration formula solved for the diagonal elements will converge. Such a matrix is called *diagonally dominant*.

Definition 5.14 (Diagonally dominant matrix)**.** An $n \times n$ matrix $\mathbf{A}$ is said to be *diagonally dominant* if its elements satisfy

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}|$$

for every $i = 1, 2, \ldots, n$.

Theorem 5.15. *If an $n \times n$ matrix $\mathbf{A}$ is diagonally dominant and $\mathbf{x}_{k+1} = \mathbf{g}(\mathbf{x})$ is the iteration formula solved with respect to the diagonal elements using the fixed-point method, then the iteration converges to the solution of the system for every initial guess $\mathbf{x}_0$.*

Proof. Omitted. □

The theorem above does not cover all possible cases where the iteration can converge, but only provides a sufficient condition for convergence. There may therefore be other types of systems for which the iteraton converges either for all or certain initial guesses.

Note that even if the coefficient matrix of the system is not diagonally dominant as such, it can in some cases be transformed into such a matrix by swapping rows. For example, for the system of equations considered earlier,

$$\begin{aligned} 4x_1 + 2x_2 + x_3 &= 8 \\ x_1 + 5x_2 + 3x_3 &= 12 \\ x_1 + x_2 + 3x_3 &= 8 \end{aligned}$$

the coefficient matrix is diagonally dominant, but this is no longer the case if the rows of the system are rearranged into any of the other six possible orders. The order of the columns in the system could also be changed, as long as corresponding changes are made to the variables' order, but this is not useful if the issue could not be resolved by rearranging the rows. To convince yourself of this, consider the following system of equations:

$$\begin{aligned} x_1 + 2x_2 + 4x_3 &= 8 \\ 2x_1 + 5x_2 + 2x_3 &= 12 \\ 3x_1 + 2x_2 + 7x_3 &= 8 \end{aligned}$$

The theorem cannot hold when the coefficient matrix is not invertible because in that case the system does not have a unique solution. It can be shown that a diagonally dominant matrix is invertible. This proof, as well as the proof of the previous theorem, is omitted.

Iteration Formula Next we derive the iteration formula of the Jacobi method in the general case. This allows the iteration formula corresponding to each system of equations to be written directly using the information obtained from the coefficient matrix A and the constant vector $\mathbf{b}$.

We attempt to solve the system of equations $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is diagonally dominant, with respect to the variables corresponding to the diagonal. These can be accessed in matrix form by decomposing the coefficient matrix into the sum of a lower triangular matrix, a diagonal matrix, and an upper triangular matrix:

$$\begin{aligned} \mathbf{Ax} &= \mathbf{b} \\ (\mathbf{L} + \mathbf{D} + \mathbf{U})\mathbf{x} &= \mathbf{b} \\ \mathbf{Lx} + \mathbf{Dx} + \mathbf{Ux} &= \mathbf{b} \\ \mathbf{Dx} &= \mathbf{b} - \mathbf{Lx} - \mathbf{Ux} \\ \mathbf{x} &= \mathbf{D}^{-1}\mathbf{b} - \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\mathbf{x}. \end{aligned}$$

Definition 5.16 (Jacobi method)**.** The iteration formula of the *Jacobi method* is of the form

$$\mathbf{x}_{k+1} = \mathbf{Bx}_k + \mathbf{c},$$

where $\mathbf{B} = -\mathbf{D}^{-1}(\mathbf{L}+\mathbf{U})$ and $\mathbf{c} = \mathbf{D}^{-1}\mathbf{b}$.

At first glance, it might seem that such an iterative method for solving a system of equations is completely unnecessary, since it requires computing the coefficient matrix of size $n \times n$, whereas the inverse matrix $\mathbf{A}^{-1}$ of size $n \times n$ would provide a direct solution. However, $\mathbf{D}$ is a diagonal matrix, so its inverse matrix can be obtained directly by taking the reciprocals of the diagonal elements, which is a computationally cheap operation.

In the implementation of the solution algorithm, it is advisable to compute the matrix $\mathbf{B}$ and the vector $\mathbf{c}$ before entering the loop to avoid unnecessary computations.

Illustration of the Convergence Criterion Let us illustrate why diagonal dominance of the coefficient matrix suffices to ensure the convergence of the iteration. It is enough to show that $\mathbf{x}_n$ converges in the case of such a system of equations.

We begin by writing out the expressions for the iterates to derive an expression for the iterate $\mathbf{x}_n$ that depends on the index n. This would allow us to analyze what happens as $n \to \infty$. Based on the iteration formula $\mathbf{x}_{k+1} = \mathbf{B}\mathbf{x}_k + \mathbf{c}$ we have

$$\begin{aligned}
\mathbf{x}_1 &= \mathbf{B}\mathbf{x}_0 + \mathbf{c} \\
\mathbf{x}_2 &= \mathbf{B}\mathbf{x}_1 + \mathbf{c} = \mathbf{B}(\mathbf{B}\mathbf{x}_0 + \mathbf{c}) + \mathbf{c} = \mathbf{B}^2\mathbf{x}_0 + \mathbf{B}\mathbf{c} + \mathbf{c} \\
\mathbf{x}_3 &= \mathbf{B}\mathbf{x}_2 + \mathbf{c} = \mathbf{B}(\mathbf{B}^2\mathbf{x}_0 + \mathbf{B}\mathbf{c} + \mathbf{c}) + \mathbf{c} = \mathbf{B}^3\mathbf{x}_0 + \mathbf{B}^2\mathbf{c} + \mathbf{B}\mathbf{c} + \mathbf{c}.
\end{aligned}$$

For the nth iteration we get

$$\mathbf{x}_n = (\mathbf{I} + \mathbf{B} + \mathbf{B}^2 + \cdots + \mathbf{B}^{n-1})\mathbf{c} + \mathbf{B}^n\mathbf{x}_0.$$

We observe that the coefficient of the vector $\mathbf{c}$ forms an expression corresponding to a geometric sum. For real numbers it holds that

$$1 + b + b^2 + \cdots + b^{n-1} = \frac{1-b^n}{1-b}$$

(Multiply the equation with the denominator and you will see why). For matrices it analogously holds that

$$\mathbf{I} + \mathbf{B} + \mathbf{B}^2 + \cdots + \mathbf{B} + \mathbf{B}^{-1} = (\mathbf{I} - \mathbf{B})^{-1}(\mathbf{I} - \mathbf{B}^n),$$

where the inverse matrix can appear on either side because, exceptionally in this particular case,

$$(\mathbf{I} + \mathbf{B} + \mathbf{B}^2 + \cdots + \mathbf{B} + \mathbf{B}^{-1})(\mathbf{I} - \mathbf{B}) = (\mathbf{I} - \mathbf{B})(\mathbf{I} + \mathbf{B} + \mathbf{B}^2 + \cdots + \mathbf{B} + \mathbf{B}^{-1}).$$

Therefore

$$\mathbf{x}_n = (\mathbf{I} - \mathbf{B})^{-1}(\mathbf{I} - \mathbf{B}^n)\mathbf{c} + \mathbf{B}^n\mathbf{x}_0.$$

For real numbers, it is well known that $b^n \to 0$ as $n \to \infty$ if $|b| < 1$ (What happens in the cases $|b| > 1$, $b = 1$, and $b = -1$?). A corresponding result holds for matrices, but this year we do not discuss matrix norms on this years. It can be shown that this always holds for the matrix $\mathbf{B}$ in the iteration formula. For example, in the case discussed above,

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_{k+1} = \begin{bmatrix} 2 \\ \frac{12}{5} \\ \frac{8}{3} \end{bmatrix} + \begin{bmatrix} 0 & \frac{2}{4} & -\frac{1}{4} \\ \frac{1}{5} & 0 & -\frac{3}{5} \\ \frac{1}{3} & -\frac{1}{3} & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_k,$$

where the coefficient matrix is diagonally dominant, we have

$$\mathbf{B} = \begin{bmatrix} 0 & \frac{2}{4} & -\frac{1}{4} \\ \frac{1}{5} & 0 & -\frac{3}{5} \\ \frac{1}{3} & -\frac{1}{3} & 0 \end{bmatrix}, \quad \mathbf{B}^2 = \begin{bmatrix} \frac{1}{60} & \frac{1}{12} & -\frac{3}{10} \\ -\frac{1}{5} & \frac{3}{10} & -\frac{1}{20} \\ -\frac{1}{15} & \frac{1}{6} & \frac{7}{60} \end{bmatrix} \quad \text{and} \quad \mathbf{B}^3 = \begin{bmatrix} -\frac{1}{12} & \frac{13}{120} & -\frac{13}{240} \\ -\frac{13}{300} & -\frac{1}{12} & -\frac{13}{100} \\ \frac{13}{180} & -\frac{13}{180} & -\frac{1}{12} \end{bmatrix}.$$

This illustrates that diagonal dominance ensures that the elements of the matrix $\mathbf{B}$ are sufficiently small, so that they decrease and approach zero when raised to higher powers. On the other hand, in the case discussed above,

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_{k+1} = \begin{bmatrix} 12 \\ 8 \\ 8 \end{bmatrix} + \begin{bmatrix} 0 & -5 & -3 \\ -1 & 0 & -3 \\ -4 & -2 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_k,$$

where the coefficient matrix is not diagonally dominant, we have

$$\mathbf{B} = \begin{bmatrix} 0 & -5 & -3 \\ -1 & 0 & -3 \\ -4 & -2 & 0 \end{bmatrix}, \quad \mathbf{B}^2 = \begin{bmatrix} 17 & 6 & 15 \\ 12 & 11 & 3 \\ 2 & 20 & 18 \end{bmatrix} \quad \text{and} \quad \mathbf{B}^3 = \begin{bmatrix} -66 & -115 & -69 \\ -23 & -66 & -69 \\ -92 & -46 & -66 \end{bmatrix},$$

so that $\mathbf{B}$ blows up when raised to higher powers.

For diagonally dominant matrices it thus holds that

$$\mathbf{x}_n = (\mathbf{I}-\mathbf{B})^{-1}(\mathbf{I}-\mathbf{B}^n)\mathbf{c} + \mathbf{B}^n\mathbf{x}_0 \xrightarrow{n\to\infty} (\mathbf{I}-\mathbf{B})^{-1}\mathbf{c},$$

which means that the iteration converges to $(\mathbf{I}-\mathbf{B})^{-1}\mathbf{c}$. We also observe that the initial guess $\mathbf{x}_0$ does not affect whether or not the iteration converges.

One might wonder why the solution is not computed directly using the formula $(\mathbf{I}-\mathbf{B})^{-1}\mathbf{c}$. The reason is the same as why the solution is not computed using the formula $\mathbf{A}^{-1}\mathbf{b}$: calculating the inverse matrix is computationally expensive.

6 Differential Equations

6.1 Euler's Method

6.2 Runge–Kutta Methods

7 Optimisation

7.1 Basic Concepts in Optimisation

7.2 Gradient Descent

7.3 Automatic Differentiation